



UNIVERSITY
OF THE PEOPLE



Interactive Sales Analytics Dashboard for Small Businesses

A Capstone Project Report of MSIT 5910

Submitted by:

Rev. Drew Brown (S551185)

For the partial fulfillment of the requirements for the degree of

Master of Science in Information Technology

Supervised by:

Dr. Sirisha Pavuluri

Department of CS & MSIT

University of the People, Pasadena, CA 91101, United States

Term 5, August, 2026



ABSTRACT

Small independent retailers hold rich transactional data but rarely have the licensing budget, the engineering capacity, or the compliance framework required to operate an enterprise business intelligence (BI) platform. This capstone project designed, implemented, and evaluated an interactive sales analytics dashboard that a small business can host on commodity hardware and maintain with a single developer. The system is a layered, modular monolith comprising 14 modules built in Python and Streamlit, using the UCI Online Retail dataset with 541,909 transaction rows. A protected pipeline validates untrusted uploads at a trust boundary, cleans the data in stages that report each row removed, pseudonymizes customer identifiers with keyed HMAC-SHA256, and stores encrypted snapshots that are decrypted only after role-based authorization, with every decision recorded in a hash-chained audit log. The methodology combined iterative prototyping with an evidence-first development workflow of protected branches, reviewed pull requests, and a continuous integration pipeline enforcing linting, formatting, secret scanning, and a coverage floor. Evaluation showed that the one-time preparation pipeline completes in 217.3 seconds at 2,494 rows per second, using 492.5 MiB of memory, that every interactive analytics query answers in under one second, and that 351 automated tests pass with 90.3 percent branch-aware coverage. The system and acceptance testing reproduced these results end-to-end on the full dataset, with all fourteen scenario cases passing. The project demonstrates that privacy-preserving, explainable analytics are affordable at small-business scale, and it contributes a teachable reference architecture that other practitioners can adapt.



UNIVERSITY
OF THE PEOPLE



ACKNOWLEDGEMENT

I would like to thank my instructor, Dr. Sirisha Pavuluri, for her guidance and feedback throughout the capstone sequence, the Department of CS & MSIT and its chair for shaping a program that made this project possible, and the University of the People for providing an accessible path to graduate study. I am also grateful to the maintainers of the open-source tools on which this project relies, and to the UCI Machine Learning Repository for stewarding the dataset that made a realistic evaluation possible.



LIST OF TABLES

Table 1: Comparison of Existing System Families Against Project Requirements.....	16
Table 2: Module Inventory by Layer and Responsibility.....	22
Table 3: Consolidated Quantitative and Qualitative Evaluation Metrics.....	34



LIST OF FIGURES

Figure 1: Dashboard Architecture.....	18
Figure 2: Snapshot Bootstrap Wiring in app.py.....	26
Figure 3: Authorized Read Path and Analysis Dispatch.....	27
Figure 4: Staged Cleaning Delta Log.....	28
Figure 5: Authentication Screen.....	29
Figure 6: Dashboard Key Performance Indicator Tab.....	30
Figure 7: Hash-Chained Audit Trail Query.....	31
Figure 8: Passing Continuous Integration Run.....	32
Figure 9: Measured Latency by Pipeline Stage and Analytics Query.....	33
Figure 10: Verified Unit and Integration Test Run With Branch-Aware Coverage.....	36
Figure 11: System and Acceptance Test Run on the Full Dataset.....	37
Figure A1: Published Releases Through v1.0.0.....	55
Figure A2: v0.1.0 Release Highlights.....	56
Figure A3: v0.9.0 Release Highlights.....	56
Figure A4: Continuous Integration History Through the v1.0.0 Release.....	57



SYMBOLS AND ABBREVIATIONS

Abbreviation	Meaning
BI	Business Intelligence
CSV	Comma-Separated Values
DFD	Data Flow Diagram
ER	Entity-Relationship
GB	Gigabyte (10^9 bytes)
GDPR	General Data Protection Regulation
HKDF	HMAC-based Key Derivation Function
HMAC	Hash-based Message Authentication Code
KPI	Key Performance Indicator
K-Means	K-Means Clustering Algorithm
MB	Megabyte (10^6 bytes)
MiB	Mebibyte (1024^2 bytes)
NIST	National Institute of Standards and Technology
OWASP	Open Worldwide Application Security Project
RFM	Recency, Frequency, and Monetary
SME	Small and Medium-Sized Enterprise
SSDF	Secure Software Development Framework
TLS	Transport Layer Security
UCI	University of California, Irvine
UML	Unified Modeling Language
XLSX	Excel Spreadsheet XML



CONTENTS

Abstract.....	2
Acknowledgement.....	3
List of Tables.....	4
List of Figures.....	5
Symbols and Abbreviations.....	6
Introduction.....	9
1.1 Background of the Study.....	9
1.2 Problem Statement.....	9
1.3 Objectives of the Study.....	10
1.4 Scope and Limitations.....	11
1.5 Report Organization.....	11
Literature Review.....	13
2.1 Overview of Related Work.....	13
Business Intelligence as a Decision Discipline.....	13
Analytics for Small and Medium Enterprises.....	13
Ethics, Privacy, and Fairness in Analytics.....	14
Software Engineering Methods and Process Evidence.....	14
2.2 Comparative Analysis of Existing Systems.....	15
2.3 Research Gap Identification.....	16
System Design.....	17
3.1 System Architecture.....	17
3.2 Methodologies and Tools Used.....	20
3.3 Data Flow and System Components.....	21
3.4 Design Diagrams (DFD, UML, ER Diagram).....	22
Results.....	24
4.1 Implementation Overview.....	24
4.2 Output and Screenshots.....	27
4.3 Evaluation Metrics.....	32



4.4 System Testing and Validation.....	35
Discussion.....	39
5.1 Interpretation of Results.....	39
5.2 Comparison with Literature Findings.....	40
5.3 Implications and Limitations.....	42
5.4 Future Work.....	43
5.5 Deployment Approach and Configuration Summary.....	44
Conclusion.....	46
6.1 Summary of Findings.....	46
6.2 Contributions.....	46
6.3 Lessons Learned and Next Improvements.....	47
References.....	49
Appendix A: Repository, Release, and Continuous Integration Evidence.....	55
Appendix B: Setup and Verification Commands.....	58



INTRODUCTION

1.1 Background of the Study

Independent retailers generate transactional data every day, but the analytical distance between that data and a defensible decision is longer than it appears. Enterprise BI platforms such as Tableau, Power BI, and Looker assume both an integration budget and a data team, neither of which the average sole proprietor possesses. Surveys of BI adoption in small and medium-sized enterprises confirm that the primary blockers are not technical capability or data availability but licensing costs, operational complexity, and the absence of an internal champion who understands both the data and the business (Tsiu et al., 2025). At the same time, the data that small retailers hold is personal data in the legal sense: a customer identifier attached to an invoice line falls within the scope of the General Data Protection Regulation in the European Union and of a growing family of state-level privacy laws in the United States (European Parliament and Council of the European Union, 2016). A small business that wants analytics, therefore, faces two simultaneous gaps, one economic and one regulatory, and tools that close the first while ignoring the second simply move the risk rather than removing it.

This capstone project starts from that double gap. It builds an interactive sales analytics dashboard that a small business can host on commodity hardware, that a single developer can maintain, and that surfaces the ethical and security implications of the decisions it supports rather than hiding them. The dashboard converts a transactional export into role-scoped views of revenue, products, customers, and time trends, and it treats pseudonymization, encryption, authorization, and audit logging as structural properties of the pipeline rather than optional features. The project was completed within the eight-week arc of the MSIT capstone course, with each unit contributing a deliverable that this report now consolidates (Oguz, 2025).

1.2 Problem Statement

A well-formed problem statement names the affected population, the evidence that the problem exists, and the consequences of leaving it unsolved (Paperpile LLC, n.d.; Research with Dr. Saeed, 2023). The problem addressed by this capstone is stated as follows. A small independent



retailer holds a transactional export of invoice-level data, typically in CSV or XLSX form. That retailer wants to know, at minimum, four things: how revenue is trending over time, which product families drive that revenue, which customer segments contribute most to it, and how the mix of the above changes seasonally. The retailer does not have a data engineer on staff and lacks the budget for Tableau, Power BI, or Looker. Because of the GDPR and growing state-level privacy legislation, the retailer cannot ignore the fact that a customer identifier is personal data. The proposed solution is a self-hosted, open-source, ethics-and-security-by-design decision-support dashboard that answers the four questions above from that transactional export, without exposing raw customer identifiers to the analytics layer.

Framing the problem this way follows the guidance in the research methods literature on evaluating candidate problems for applied computing projects: the problem is current, it is bounded enough to be addressed by a single developer within one term, and its solution produces both a working artifact and transferable knowledge (PhD Assistance, 2020; Sheppard, 2019). The problem is also verifiable because each of the four business questions can be answered, measured, and inspected against the dataset described below.

1.3 Objectives of the Study

The project pursued five objectives, which together operationalize the problem statement into requirements in the manner recommended for software requirements specifications (Krüger, 2025). The first objective was to design a modular architecture whose security and ethics obligations are attached to named enforcement points rather than aggregated into aspirational prose. The second was to implement a protected data pipeline in which untrusted files are validated at a trust boundary, cleaned in auditable stages, pseudonymized before storage, and encrypted at rest. The third was to implement decision-support analytics, including revenue, product, geographic, and time-series metrics and a fairness-aware recency, frequency, and monetary segmentation, that read only the protected snapshot. The fourth was to present those analytics through an authenticated, role-scoped Streamlit interface, where each chart explains the formula, filters, exclusions, and row count behind it. The fifth was to demonstrate process discipline appropriate to a production system, including version control with protected branches, reviewed pull requests, continuous integration gates, comprehensive testing, and published releases.

The dataset used as a proxy for a small retailer export is the UCI Online Retail dataset, which contains 541,909 invoice-line records from a UK-based online retailer between December 2010 and December 2011 (D. Chen, 2015). The dataset is released under the Creative Commons Attribution 4.0 license, which permits academic reuse with attribution. Its schema includes invoice number, stock code, product description, quantity, invoice date, unit price, customer identifier, and country, and its size of about 23 MB sits comfortably within the working set of a laptop.

1.4 Scope and Limitations

Scope was fixed early using the discipline of a feasibility assessment: technical feasibility was bounded by what one developer could build and operate in eight weeks, economic feasibility by a zero licensing budget, and operational feasibility by the capacities of a non-technical small-business audience (NxtChair, 2024). Within scope are the fourteen numbered modules of the layered architecture, the four dashboard views, three user roles, the protected pipeline, the audit surface, and the testing and release evidence. Out of scope are streaming ingestion, multi-node deployment, a controlled user study with real retail managers, and machine-learning methods beyond K-Means segmentation, each of which is discussed as future work in Chapter 5.

Three limitations qualify the results reported here. First, the evaluation used a single historical dataset, so behavior on streaming data, larger volumes, or differently shaped schemas remains unmeasured. Second, persistence relies on SQLite and encrypted local files, which makes the current design a single-instance design by construction. Third, the usability findings come from a heuristic inspection rather than from observing real managers, so they identify likely problems without measuring task success. These limitations are honest boundaries rather than defects, and the deliberate scoping that produced them is itself a guard against the optimism bias that inflates project plans (Flyvbjerg, 2008).

1.5 Report Organization

The report follows the structure recommended for capstone project documents (Miller, 2026). Chapter 1 has established the background, problem, objectives, and scope. Chapter 2 reviews the literature on BI, small-business analytics, fairness and privacy, and the software engineering practices adopted by the project, and identifies the gap the project fills. Chapter 3 presents the system design: layered architecture, methodologies and tools, data flow through the



fourteen modules, and design diagrams. Chapter 4 presents the results, including implementation, output and screenshots, evaluation metrics, and system testing that validated the integrated whole. Chapter 5 interprets the results against the literature, states implications and limitations, describes the deployment approach, and proposes future work. Chapter 6 concludes with a summary of findings, contributions, and lessons learned. References and appendices with supporting evidence complete the document.



LITERATURE REVIEW

2.1 Overview of Related Work

The literature that most directly informs this project sits at the intersection of four streams: BI as a decision-support discipline, analytics adoption in small and medium-sized enterprises, ethics and privacy in data-intensive software, and software engineering methods that enable a single developer to build a defensible system.

Business Intelligence as a Decision Discipline

H. Chen et al. (2012) defined the modern shape of the BI and analytics field and organized it into three evolutionary stages, culminating in what they called BI&A 3.0, which later authors now call embedded analytics. Their framing is important for this capstone because it makes clear that a dashboard is not an end in itself; it is a delivery vehicle for decision-making. Yigitbasioglu and Velcu (2012) reinforce this in their review of dashboard design as a performance-management tool, arguing that the quality of a dashboard should be measured not by the number of visuals it renders but by the clarity of the decision it supports. The design in this capstone treats that clarity as a first-class functional requirement: every chart is annotated with the aggregation, the applied filter, and the exclusions that produced it.

Analytics for Small and Medium Enterprises

Tsiu et al. (2025) surveyed the adoption of BI tools by small and medium enterprises and found that the primary blockers are not technical capability or data availability but licensing costs, operational complexity, and the absence of an internal champion who understands both the data and the business. Their findings shape three design decisions in this capstone. First, the entire stack is open source and self-hostable, so licensing cost is zero. Second, the deployment is a single Python process under Streamlit, so operational complexity is bounded by what one developer can hold in their head. Third, the four views the dashboard exposes align with the questions a small-business owner asks in weekly practice, reducing the need for a translation layer between the analyst and the business. John et al. (2023) provide a complementary perspective by examining clustering algorithms on transactional retail data from the same UK market that produced the



project dataset. Their analysis of RFM segmentation as a low-cost, high-clarity method supports the choice of RFM plus K-Means clustering over a more elaborate deep-learning approach: K-Means is transparent, its cluster centers can be explained in plain language to a non-technical stakeholder, and its sensitivity to input choices can be documented and tested.

Ethics, Privacy, and Fairness in Analytics

Mehrabi et al. (2021) survey the taxonomy of bias and fairness in machine learning and identify segmentation-driven benefit concentration as one of the harms most easily missed in dashboard-style deployments. Their taxonomy motivates the fairness-sensitivity check embedded in the segmentation module: cluster labels are recomputed on a holdout set to confirm stability, and analytic outcomes are suppressed for cohorts of fewer than five customers to prevent re-identification of individuals through their cluster labels. On the security side, the OWASP Foundation (2025) publishes the OWASP Top Ten, which serves as the foundation for the threat models at the presentation and ingestion layers. Souppaya et al. (2022) codify NIST SSDF v1.1 as a small, actionable set of secure-development practices that a single developer can implement without an enterprise security team. The GDPR establishes privacy by design and data minimization as legal obligations in the European Union (European Parliament and Council of the European Union, 2016), and the NIST Privacy Framework provides a compatible control catalog on the United States side (National Institute of Standards and Technology, 2020).

Software Engineering Methods and Process Evidence

Walker (2022) surveys fourteen software architecture patterns and characterizes the modular monolith as an accessible starting point for teams that cannot yet justify the operational costs of microservices, a characterization that video treatments of architecture fundamentals echo for newcomers to the field (A Dev' Story, 2020; ByteMonk, 2024). Davies (2024) formalizes the discipline of expressing architecture as blueprints that another engineer can read, Letaw (2024) provides the software engineering textbook framing used to structure the module list, and Summers (2020) supplies the effective-methods vocabulary for aligning requirements, design, and testing. Guidance on software design documents stresses that a design document should record not only the chosen structure but the reasoning and the rejected alternatives (Bit.ai Editorial Team, n.d.; Lucid Software, n.d.), and the documentation literature adds that such records succeed when they are task-oriented, current, and located where the reader already is (Brown, 2026). Bergmann



(2021) surveys open-source project management tooling and motivates the choice of GitHub Issues and Projects as the coordination layer for a single-developer capstone, while practitioner tutorials on Git, GitHub, branching, and release tagging describe the version control vocabulary the project adopts as evidence (Corbin, 2025; SupersimpleDev, 2021; Swiftful Thinking, 2024; Tech With Diego, 2025). The DevOps literature frames continuous integration and delivery as the cultural and technical practice that keeps such evidence trustworthy (Narasimhan et al., 2023; Parzych, 2019; Red Hat, 2025; Simplilearn, 2021).

2.2 Comparative Analysis of Existing Systems

Existing systems that a small retailer might adopt fall into three families, and each fails a requirement this project treats as non-negotiable. The first family is the enterprise BI platform, exemplified by Tableau, Power BI, and Looker. These platforms are mature and visually rich, but their licensing models and integration assumptions place them beyond the reach of the audience identified by Tsiu et al. (2025), and their compliance posture is configured by specialists the small retailer does not employ. Managed DevOps suites such as Azure DevOps illustrate the same pattern on the process side: they are powerful but assume an organization, not an individual (Knowledge Academy, 2023). The second family is the spreadsheet, which is affordable and familiar but offers no role separation, no audit trail, no pseudonymization, and no protection against the formula-injection class of attacks that a shared workbook invites (OWASP Foundation, 2025). The third family is the custom script, a one-off Python or R analysis that answers a question once but preserves no provenance, enforces no access control, and decays as soon as its author moves on.

Table 1 compares these families against the requirements of the problem statement. The comparison makes the gap concrete: no existing option combines zero licensing cost, single-developer operability, privacy protection by construction, and decision-grade explainability. Open-source dashboard frameworks such as Streamlit and Dash provide the presentation substrate for closing the gap, but a framework is not a system; the protected pipeline, the role model, the audit surface, and the testing discipline still have to be designed and built, which is precisely the work this capstone reports.



Table 1

Comparison of Existing System Families Against Project Requirements

Requirement	Enterprise BI platforms	Spreadsheets	Custom scripts	This project
Licensing cost within a small-business budget	No	Yes	Yes	Yes
Operable by a single non-specialist developer	No	Yes	Partly	Yes
Pseudonymization of customer identifiers	Configurable	No	Rarely	Yes, by construction
Role-based access control	Yes	No	No	Yes
Tamper-evident audit trail	Partly	No	No	Yes
Explainable chart provenance	Partly	No	No	Yes
Automated quality gates and tests	Vendor-managed	No	Rarely	Yes

2.3 Research Gap Identification

Reading the four streams together exposes the gap this project fills. The BI literature establishes what a decision-support dashboard should do, and the SME literature establishes why small firms cannot buy that capability off the shelf. The fairness and privacy literature establishes obligations that apply regardless of firm size, and the software engineering literature provides methods that enable a defensible build. What the literature does not supply is a worked, teachable example that integrates all four: a small-scale reference architecture in which privacy protection, access control, audit logging, explainability, and process evidence are implemented together, measured honestly, and documented at a level of detail another student or practitioner can reproduce. Existing academic treatments tend to demonstrate one concern in isolation, and existing commercial systems hide their integration behind licensing. This capstone addresses the gap by building and evaluating an example of exactly this kind and by reporting the measurements, trade-offs, and failures alongside the successes.

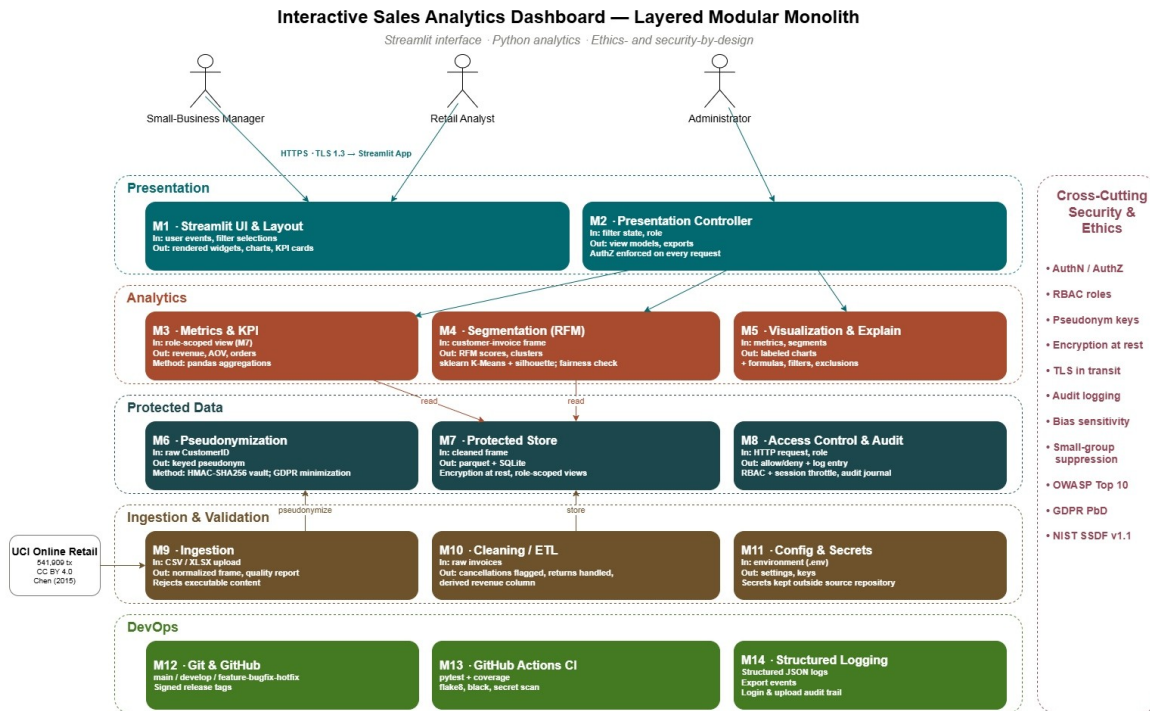


SYSTEM DESIGN

3.1 System Architecture

The design that emerged from Unit 3 is a layered modular monolith with five horizontal layers and a cross-cutting Security and Ethics band. The architectural style was chosen deliberately. Walker (2022) characterizes the modular monolith as an accessible starting point that preserves the option of a later split into services without paying the operational cost up front, and introductory treatments of software architecture stress that the first duty of an architecture is to make responsibilities and boundaries legible (A Dev' Story, 2020; ByteMonk, 2024). This capstone is unlikely to reach the scale where a microservice split is justified, so the design invests instead in strict module contracts inside a single process. Figure 1 shows the full architecture, and the sections below walk through each layer.

Figure 1
Dashboard Architecture



Note. The layered modular monolith groups fourteen modules into presentation, analytics service, protected data, ingestion and validation, and DevOps and observability layers, with a cross-cutting Security and Ethics band on the right.

The presentation layer contains modules M1 and M2 and is implemented in Streamlit, the smallest reasonable framework that produces a professional, role-scoped, accessible interactive dashboard from Python code alone. Module M1 renders the interface: the sidebar, the four views covering revenue, products, customers, and time trends, the KPI cards, and the exports. Module M2 is a thin controller that reads the authenticated session, asks module M8 to authorize the request against the requested view, and hands off to the analytics service. Every call from M2 downward passes through the Security and Ethics band, so no code path in the presentation layer touches raw data.

The analytics service layer contains modules M3, M4, and M5. Module M3 computes the key performance indicators, including revenue, average order value, order count, and product mix, as pandas aggregations against the role-scoped view served by M7. Module M4 computes RFM scores and clusters using scikit-learn K-Means with a silhouette score, and it performs a fairness



sensitivity check by removing a small holdout and confirming that cluster assignments are stable. Module M5 renders every chart with an inline caption that documents the aggregation, the applied filter, and any exclusions, such as null customer identifiers or small-group suppression. Nothing in this layer holds mutable state between requests; every view is idempotent given a role, a filter, and a snapshot identifier.

The protected data layer contains modules M6, M7, and M8 and is where the design is deliberately opinionated. Module M6 replaces the raw customer identifier with a keyed HMAC-SHA256 pseudonym at the moment the frame crosses into the protected store. The HMAC key is stored in an environment-scoped vault, never logged, and rotated quarterly. Because HMAC is deterministic per key, downstream grouping and RFM aggregation still work, but the pseudonym is one-way and cannot be inverted without the key. Module M7 persists the pseudonymized frame as an encrypted Parquet snapshot and exposes role-scoped views to the analytics layer. Module M8 is the single authorization checkpoint for the application: every request passes through it, every allow and every deny are written to the audit journal, and the small-group suppression threshold of five customers is enforced here rather than left to the presentation layer.

The ingestion and validation layer contains modules M9, M10, and M11, described in detail in Section 3.3. The DevOps and observability layer contains modules M12, M13, and M14. Module M12 is the Git repository itself, module M13 is the GitHub Actions continuous integration pipeline, which runs pytest with a coverage floor, flake8, black, and a secret scan on every push and pull request, and module M14 emits structured JSON logs for every login, upload, export, and authorization decision. The DevOps layer is the audit surface that lets a reviewer confirm, after the fact, that a given data view was requested by a given role at a given time and that the request was authorized.

The band on the right of Figure 1 is not a layer in the classical sense; it is the set of concerns that every layer inherits. Authentication and role-based authorization are enforced at M2 and M8. Pseudonymization keys live in M11 and are consumed by M6. Encryption at rest is a property of M7. TLS in transit terminates at M1. Audit logging is the responsibility of M14. M4 and M8 enforce bias sensitivity and small-group suppression. The three external reference frameworks, namely the OWASP Top Ten (OWASP Foundation, 2025), NIST SSDF v1.1 (Souppaya et al., 2022), and GDPR privacy by design (European Parliament and Council of the

European Union, 2016), are stitched into these enforcement points rather than left as aspirational text at the end of the document.

3.2 Methodologies and Tools Used

The development methodology combined an iterative prototyping model with an evidence-first engineering process. The prototyping model fits projects whose requirements are clear in outline but negotiable in detail: an early working version is built, evaluated, and refined in cycles rather than specified exhaustively in advance (GeeksforGeeks, 2025a; Laptick, 2022). The capstone followed that arc across the term. Weeks 1 through 3 established the purpose, the proposal, and the architecture; Weeks 4 and 5 implemented the ingestion, cleaning, configuration, pseudonymization, and storage modules; Week 6 implemented the analytics, access control, and interface wiring; and Weeks 7 and 8 delivered testing, maintenance planning, and this report. Each cycle ended with a reviewable artifact, which kept the prototype honest in the way the effective-methods literature recommends, with requirements, design, and verification advancing together rather than in sequence (Letaw, 2024; Summers, 2020). Requirements themselves were managed in the spirit of a lightweight software requirements specification, with each objective stated verifiably and traced to modules and tests (Krüger, 2025).

The design was recorded before it was built. A design document following the practices recommended for such documents captured the module list, the layer boundaries, the threat model, and the rejected alternatives, so that the architecture existed as a blueprint another engineer could read before any module was implemented (Bit.ai Editorial Team, n.d.; Davies, 2024; Lucid Software, n.d.).

The tool selection is deliberately small. Python 3.12 is the implementation language. Streamlit provides the presentation layer, streamlit-authenticator the login surface, pandas and NumPy the data manipulation, scikit-learn the K-Means segmentation, Plotly the charts, PyArrow and Parquet the snapshot format, the cryptography library the Fernet encryption and HKDF key derivation, SQLAlchemy and SQLite the audit and credential store, and python-dotenv the configuration seam. The development-side tools include pytest with coverage for testing, flake8 and black for style and formatting, gitleaks for secret scanning, and GitHub Actions for continuous integration, reflecting standard DevOps practices scaled to a single developer (Narasimhan et al., 2023; Parzych, 2019; Red Hat, 2025; Simplilearn, 2021). Project coordination uses GitHub Issues



and Projects (Bergmann, 2021), and version control follows a Git Flow branching model with protected main and develop branches, feature branches, reviewed pull requests, conventional commit messages scoped to module numbers, and annotated release tags (Corbin, 2025; SupersimpleDev, 2021; Swiftful Thinking, 2024; Tech With Diego, 2025).

3.3 Data Flow and System Components

Data enters the system exactly once, at module M9, and every downstream module consumes the output of the module before it. M9 is the trust boundary between the outside world and the rest of the system. It accepts a transactional export from the retailer, confirms that the file is what it claims to be, and rejects any content that could execute or exceed safe resource bounds when the file is opened. Its design goal is to treat hostile or malformed input as a rejection event rather than a debugging event: any failure of trust is raised as a typed error, and any lesser data-quality concern is recorded as a severity-tagged issue in the data-quality report accompanying the accepted frame. The rejection policy is drawn from OWASP A05, covering injection, and A08, covering software and data integrity failures, and it addresses the standard families of executable content in spreadsheet exports, along with encrypted files, files that mis-declare their type, and files large enough to threaten availability (OWASP Foundation, 2025). When a file is accepted, M9 hands the pipeline a schema-valid, unmutated frame, along with a machine-readable report of any anomalies observed during validation.

Module M10 is where a schema-valid frame becomes an analysis-ready frame. It runs a small set of clearly separated transformations in a fixed order so that each step is auditable in isolation. First, columns are coerced to their intended types, with the customer identifier retained in a form that survives spreadsheet round-trips and can be pseudonymized deterministically by M6. Second, rows that cannot be associated with a customer are removed, because a row without a customer identifier cannot be carried across the pseudonymization boundary without introducing an ambiguous null class inside the protected store. Third, exceptional invoice classes are recognized: cancellations, bookkeeping adjustments, and product returns are each identified by explicit rules rather than loose prefix matching, and eligible returns are paired with their originating invoices, so the pairing rate itself becomes an observable quality signal. Fourth, revenue is derived from quantity and unit price. Fifth, non-product invoice codes covering postage, bank charges, and similar administrative entries are tagged rather than deleted, so that revenue

reconciles across the frame while product-mix views can exclude them cleanly. Every transformation reports the rows it reads, the rows it produces, and a small set of counters that explain the differences, and these deltas flow into the structured audit stream owned by M14. Module M11 loads configuration from an environment file that is never checked into the repository and exposes an immutable settings object to the rest of the application, ensuring that secrets are held in exactly one place and that the running configuration cannot be mutated by later code.

From M10, the frame crosses the protected boundary: M6 pseudonymizes the customer identifier column in place, M7 encrypts and persists the snapshot, and all subsequent reads flow through M8 authorization into M3 metrics, M4 segmentation, and M5 chart construction, with M1 and M2 presenting the results. Table 2 summarizes the fourteen modules, their layers, and their responsibilities.

Table 2
Module Inventory by Layer and Responsibility

Module	Layer	Responsibility
M1	Presentation	Render sidebar, tabs, KPI cards, and exports
M2	Presentation	Route authenticated requests through authorization
M3	Analytics service	Compute revenue, product, country, and repeat-rate metrics
M4	Analytics service	Compute RFM K-Means segments with stability check
M5	Analytics service	Attach formula, filters, exclusions, and row counts to every chart
M6	Protected data	Pseudonymize customer identifiers with keyed HMAC-SHA256
M7	Protected data	Persist and decrypt encrypted Parquet snapshots
M8	Protected data	Authorize every read and enforce small-group suppression
M9	Ingestion and validation	Validate uploads at the trust boundary and reject unsafe content
M10	Ingestion and validation	Clean the frame in staged, delta-reporting transformations
M11	Ingestion and validation	Load and validate immutable configuration and secrets
M12	DevOps and observability	Version control repository and branching evidence
M13	DevOps and observability	Continuous integration quality gates
M14	DevOps and observability	Structured logging and hash-chained audit journal

3.4 Design Diagrams (DFD, UML, ER Diagram)

The architecture diagram in Figure 1 doubles as a level-one data flow diagram, because the layered design constrains data to a single path: an untrusted file flows through validation, cleaning, and pseudonymization into the encrypted store, and role-scoped reads flow out of the



store through authorization into metrics and charts. Two design decisions from Week 3 merit specific comment because they shaped the diagram. First, the ethics and security band was elevated to a first-class element of the drawing rather than a paragraph in the design section, so that reviewers see the enforcement points at the same visual weight as the functional modules. Second, the repository scaffold was generated directly from the diagram, with one source module per numbered box, a test module mirroring each source module, and a continuous integration pipeline guarding the whole, keeping the blueprint and the code aligned, as Davies (2024) argues blueprints must. The entity structure of the protected store is deliberately flat: a single pseudonymized invoice-line table per snapshot, plus the audit event and credential tables that module M14 owns in SQLite, which are migrated via versioned schema scripts. A conventional entity-relationship elaboration adds little at this scale, and the flat snapshot design is what makes idempotent rebuilds and image-tag rollbacks trustworthy.



RESULTS

4.1 Implementation Overview

The delivered artifact is an interactive sales analytics dashboard that converts the UCI Online Retail transaction dataset into protected, manager-facing insight for small businesses (D. Chen, 2015). The implementation follows the architecture specified in Chapter 3: a layered modular monolith with fourteen modules, M1 through M14, grouped into presentation, analytics, protected data, ingestion, and cross-cutting DevOps responsibilities. Presentation consists of the M1 tabbed dashboard shell, the M2 authentication and data request interface, and the M5 explainable visualization layer. Analytics consists of the M3 key performance indicator functions and the M4 recency, frequency, and monetary segmentation model. Protected data consists of M6 pseudonymization, M7 encrypted snapshot storage, and M8 role-based access control. Ingestion consists of the M9 validation trust boundary and the M10 staged cleaning pipeline. The cross-cutting modules are the M11 immutable configuration object, the M13 continuous integration and delivery workflow, and the M14 hash-chained audit log.

The core of the implementation is a protected pipeline that executes in a single pass and leaves an audit trail at every step. M9 acts as a trust boundary rather than a convenience loader: it checks declared schema, media type, file size, and unsafe content patterns, such as macros and spreadsheet formula injection, before any untrusted bytes reach a data frame. M10 then applies staged cleaning and reports a row delta for each stage, rather than silently discarding records, which converts data preparation from an opaque step into a measurable one. M6 rewrites the customer identifier column in place using keyed HMAC-SHA256, so that no downstream module, log, or chart ever holds a natural identity key. M7 persists the pseudonymized frame as a Fernet-encrypted Parquet snapshot, whose encryption key is derived using HKDF-SHA256 from the configured secret. M8 authorizes every read of that snapshot against the acting role, and M14 records each authorization decision as a hash-chained event in a SQLite audit database, so that tampering with the history of access breaks the chain and becomes visible.



The analytics layer consumes only the protected snapshot. M3 computes a revenue summary, product metrics, a monthly and weekday time series, country-level revenue, and a repeat purchase rate from an eleven-column input contract that fails fast when the contract is violated, preventing malformed frames from producing plausible but incorrect figures. M4 fits a K-Means model on recency, frequency, and monetary features with a fixed random seed so that the same input always yields the same segments, checks segment stability with an adjusted Rand index comparison against a held-out fraction of customers, and suppresses any segment smaller than the configured small-group threshold so that a cluster cannot become a reidentification vector. M5 wraps every figure in an explained chart object that includes the formula used, the filters applied, the exclusions made, and the row count behind the visual, giving the dashboard reader the same provenance information that a code reviewer would expect.

Integration is concentrated in the application module, which imports all backend modules and controls the sequence in which they run. The bootstrap function is idempotent: it looks for an existing snapshot and, only when none is found, ingests, cleans, pseudonymizes, and writes a snapshot identified by its coordinated universal time stamp, emitting a protected store write event as it does so. Because the bootstrap is the only writer, the expensive preparation work happens once rather than on each interaction, and because it is idempotent, a restarted container reuses the existing encrypted snapshot rather than rebuilding it. Figure 2 shows the bootstrap wiring as implemented.

**Figure 2***Snapshot Bootstrap Wiring in app.py*

```
def _ensure_snapshot(
    settings: Settings,
    *,
    source: Path,
) -> str:
    """Return the latest snapshot id, creating one from source if needed.

    Existing snapshots are reused. When none exist, source is ingested, cleaned, pseudonymized,
    and stored as a timestamped snapshot.
    """

    existing = list_snapshots(settings)
    if existing:
        return existing[-1]

    raw = ingest(source)
    cleaned = clean(raw)
    key = settings.require_pseudonym_key().encode("utf-8")
    pseudonymized = pseudonymize_column(cleaned, key=key, drop_source=True)

    snapshot_id = datetime.now(timezone.utc).strftime("%Y%m%dT%H%M%S")
    audit_log = AuditLog(Database(settings))
    write_snapshot(
        pseudonymized,
        settings,
        snapshot_id=snapshot_id,
        audit_log=audit_log,
    )
    return snapshot_id
```

Note. The excerpt shows the idempotent bootstrap that ingests, cleans, pseudonymizes, and writes an encrypted snapshot, recording a protected store write event via the audit log.

The second integration decision that shapes the whole system is that authorization precedes decryption. The read path in the storage module authorizes the acting role against the requested view, writes the decision to the audit log, raises a storage error when the decision is a denial, and only then decrypts the snapshot. Ordering the checks this way means that a denied request never causes protected plaintext to exist in memory, which is a stronger guarantee than filtering results after the fact. Presentation-level routing is equally explicit: the tab list is derived from the authenticated role, the analyses offered within a tab are derived from the tab, and a dispatch function maps each analysis identifier to a specific metric and chart pair for revenue summary indicators, revenue by month, revenue by weekday, top products, country revenue, repeat



rate, and customer segments. An unknown identifier raises a value error rather than returning an empty figure, so a wiring mistake fails loudly during development rather than appearing as a blank panel in production. Configuration flows through the same seam: an immutable settings object is injected into every module, and invalid configuration raises a configuration error at construction time. Figure 3 shows the authorized read path and the dispatch table together.

Figure 3

Authorized Read Path and Analysis Dispatch

```
src > storage.py > ...
267 def read_view(
268     view: ViewName,
269     actor: Actor,
270     settings: "Settings",
271     *,
272     snapshot_id: str | None = None,
273     audit_log: AuditLog | None = None,
274 ) -> "pd.DataFrame":
275     """Return a role-scoped view of a snapshot.
276
277     M8 authorizes the actor and view before anything is decrypted. If access is denied,
278     StorageError is raised. If access is allowed, the snapshot is decrypted and filtered
279     by role and view before being returned.
280     """
281
282     if view not in _VIEW_COLUMNS:
283         raise StorageError(f"unknown view: {view!r}")
284
285     log = audit_log if audit_log is not None else AuditLog(Database(settings))
286     try:
287         decision = authorize(actor, view, settings, audit_log=log)
288     except AccessError as exc:
289         raise StorageError(str(exc)) from exc
290
291     if not decision.allowed:
292         raise StorageError(f"role {actor.role!r} is not permitted to read view {view!r}")
293
294     frame = read_snapshot(settings, snapshot_id=snapshot_id)
295     return _apply_view_filters(frame, actor.role, view)

src > app.py > ...
315
316
317 def dispatch_analysis(
318     analysis_id: str,
319     frame: pd.DataFrame,
320     *,
321     settings: Settings,
322 ) -> ExplainedChart:
323     """Route an analysis id to its metric and chart renderer.
324
325     Unknown ids raise ValueError.
326     """
327
328     if analysis_id == "revenue_summary_kpis":
329         return _render_revenue_summary_kpis(frame)
330     if analysis_id == "revenue_by_month":
331         return _render_revenue_by_month(frame)
332     if analysis_id == "revenue_by_weekday":
333         return _render_revenue_by_weekday(frame)
334     if analysis_id == "top_products":
335         return _render_top_products(frame)
336     if analysis_id == "country_revenue":
337         return _render_country_revenue(frame)
338     if analysis_id == "repeat_rate":
339         return _render_repeat_rate(frame)
340     if analysis_id == "customer_segments":
341         return _render_customer_segments(frame, settings)
342     raise ValueError(f"unknown analysis id: {analysis_id!r}")
343
```

Note. The excerpt shows role-based authorization and audit logging execution before decryption, and the dispatch function that binds each tab analysis identifier to one metric and chart pair.

4.2 Output and Screenshots

The end-to-end behavior of the system was exercised on the real dataset rather than on synthetic samples. The input is the Online Retail workbook placed in the raw data directory, an approximately 23 MB spreadsheet containing 541,909 transaction rows. M9 accepted the workbook after schema, media type, size, and unsafe content checks, and M10 then reported its work stage by stage. Type coercion passed 541,909 rows through unchanged while flagging 135,080 rows whose customer identifier was missing. The null customer stage removed exactly those flagged rows, reducing the frame from 541,909 to 406,829 rows. Cancellation tagging identified 8,905 cancellation and return rows, of which 2,787 could be paired with an originating sale and 6,118 could not. Revenue derivation produced 8,905 negative revenue rows and 40 zero revenue rows, and non-product tagging marked 1,786 rows as postage, fees, and similar non-



merchandise lines. The final protected frame, therefore, contains 406,829 clean rows, each carrying an explicit set of tags that later analytics can intentionally include or exclude.

Reporting these deltas in the log rather than in a comment is what makes the output defensible. A reader can reconcile the difference between the 541,909 rows that entered the pipeline and the 406,829 rows that reached analytics by adding the reported removals, and can see that no stage quietly changed the population. The tagged rather than deleted approach to cancellations, zero revenue lines, and non-product lines preserves the option to analyze returns later while keeping default revenue figures free of distortion. Figure 4 shows the stage delta log for the verified run.

Figure 4

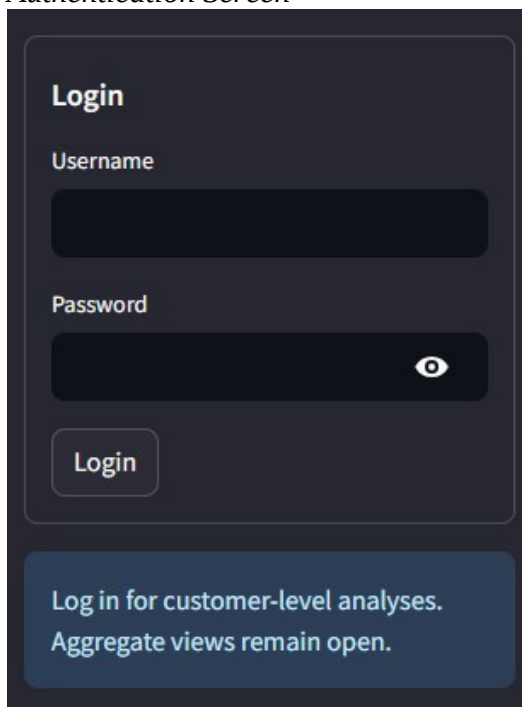
Staged Cleaning Delta Log

```
{
  "ts": "2026-07-28T18:10:16+00:00",
  "level": "INFO",
  "logger": "src.ingestion",
  "message": "ingest start source_id=91ec1df8b43a kind=xlsx"
}
{
  "ts": "2026-07-28T18:10:16+00:00",
  "level": "INFO",
  "logger": "src.ingestion",
  "message": "ingest ok source_id=91ec1df8b43a rows=541909 accepted=541909 errors=0 warnings=0"
}
{
  "ts": "2026-07-28T18:11:29+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean stage=type_coercion in=541909 out=541909 details={
    'quantity_missing': 0, 'quantity_uncoerced': 0, 'unit_price_missing': 0,
    'unit_price_uncoerced': 0, 'customer_id_missing': 135080, 'customer_id_uncoerced': 0,
    'invoice_date_missing': 0, 'invoice_date_uncoerced': 0
  }"
}
{
  "ts": "2026-07-28T18:11:56+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean stage=null_customer_drop in=541909 out=406829 details={
    'customer_id_rows_dropped': 135080
  }"
}
{
  "ts": "2026-07-28T18:11:56+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean stage=cancellation_tagging in=406829 out=406829 details={
    'cancellations': 8905, 'adjustments': 0, 'returns': 8905, 'paired_cancellations': 2787,
    'unpaired_cancellations': 6118, 'paired_returns': 2787, 'unpaired_returns': 6118,
    'pairing_ineligible_returns': 0, 'pairing_ineligible_originals': 0,
    'cancellations_without_negative_quantity': 0
  }"
}
{
  "ts": "2026-07-28T18:11:56+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean stage=revenue_derivation in=406829 out=406829 details={
    'negative_revenue_rows': 8905, 'zero_revenue_rows': 40
  }"
}
{
  "ts": "2026-07-28T18:11:56+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean stage=non_product_tagging in=406829 out=406829 details={
    'non_product_rows': 1786
  }"
}
{
  "ts": "2026-07-28T18:11:56+00:00",
  "level": "INFO",
  "logger": "src.cleaning",
  "message": "clean done input=541909 output=406829 dropped=135080"
}
```

Note. The log records the reduction from 541,909 input rows to 406,829 clean rows, including 135,080 rows removed for missing customer identifiers and the tagging of 8,905 cancellation and return rows.

After cleaning, the pipeline pseudonymized the customer identifier column and wrote the encrypted snapshot. The dashboard itself never reads the raw workbook. When a user authenticates through the M1 interface, the resolved role determines which tabs appear, and each tab requests analyses from the snapshot through the authorized read path. An analyst sees revenue, product, geography, and segmentation views; a viewer sees a reduced set; and an administrator additionally sees operational information. Every chart displays its formula, filters, exclusions, and the number of rows behind it, so the interface states not only what the number is but how it was produced. Figures 5 and 6 show the authentication screen and the key performance indicator tab, and Figure 7 shows an audit trail query returning the hash-chained record of reads and writes performed during the run.

Figure 5
Authentication Screen

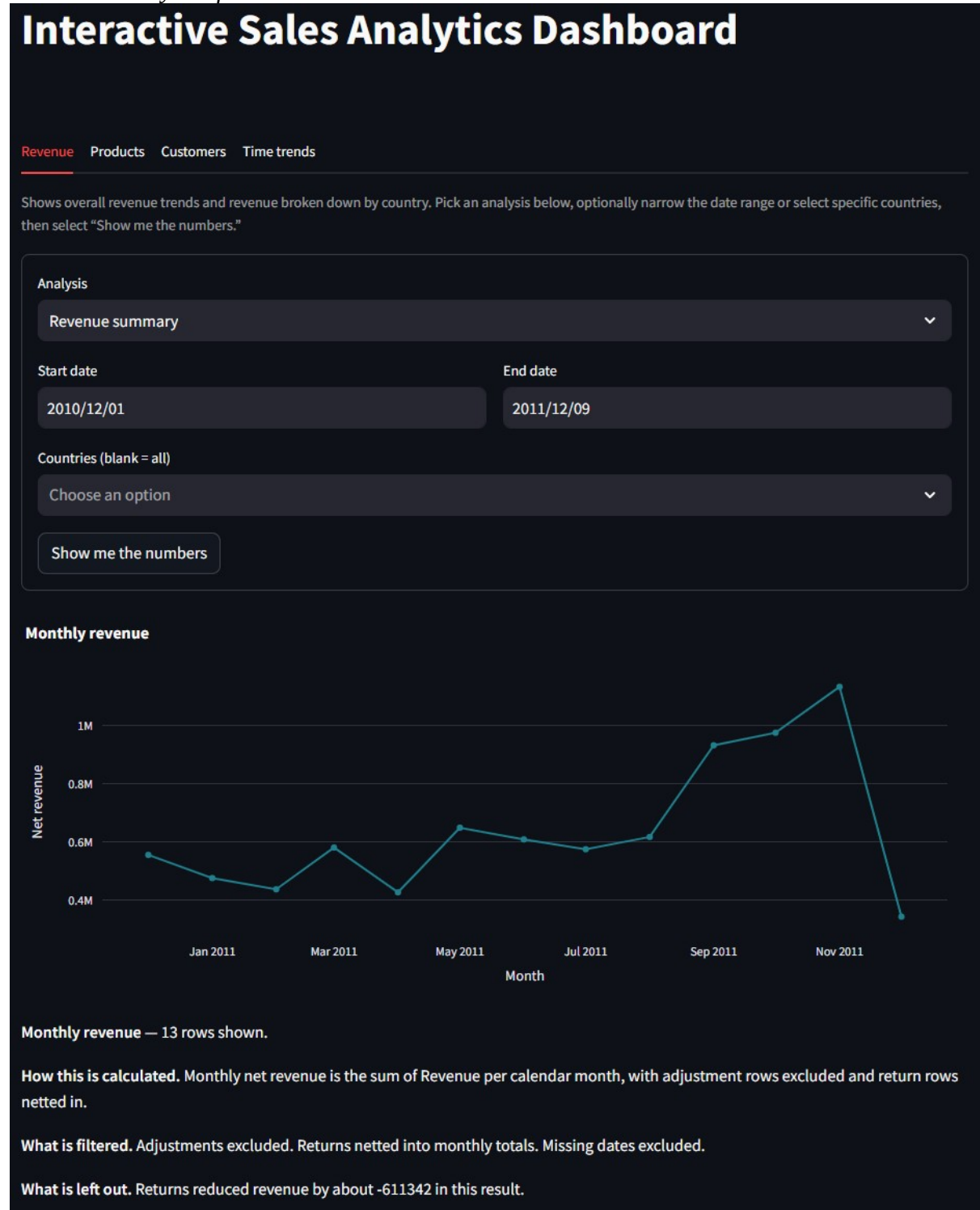
The image shows a dark-themed login interface. At the top, the word "Login" is displayed in a light blue font. Below it, there are two input fields: "Username" and "Password". The "Password" field has a small eye icon to its right, indicating a toggle for visibility. Below the input fields is a "Login" button. At the bottom of the form, there is a blue box containing the text "Log in for customer-level analyses. Aggregate views remain open."

Note. The login screen collects credentials through streamlit-authenticator, and the resolved role determines which dashboard tabs and analyses become available.



Figure 6

Dashboard Key Performance Indicator Tab



Note. Each figure carries its formula, filters, exclusions, and row count, so that displayed indicators can be traced to the transformation that produced them.



Figure 7
Hash-Chained Audit Trail Query

timestamp	actor	action	hash	prev_hash
2026-07-28T18:11:58+00:00	pipeline	protected_store_write	2acf986531db1a0	NULL
2026-07-28T18:11:59+00:00	admin@local	protected_store_read	7701c475bf4ea72	2acf986531db1a0
2026-07-28T18:12:00+00:00	admin@local	protected_store_read	0aec23297a5635b	7701c475bf4ea72
2026-07-28T18:12:00+00:00	admin@local	protected_store_read	725e26f260d38c6	0aec23297a5635b
2026-07-28T18:16:05+00:00	admin@local	protected_store_read	c99cbbcce93b4a3	725e26f260d38c6
2026-07-28T18:16:06+00:00	admin@local	protected_store_read	e70e01af28f931d	c99cbbcce93b4a3
2026-07-28T18:16:06+00:00	admin@local	protected_store_read	fac6f46cf571074	e70e01af28f931d
2026-07-28T18:16:06+00:00	admin@local	protected_store_read	f4bf4973f666504	fac6f46cf571074
2026-07-28T18:16:07+00:00	admin@local	protected_store_read	156bd2dcef0767a	f4bf4973f666504
2026-07-28T18:16:08+00:00	admin@local	protected_store_read	77bf520b06b7590	156bd2dcef0767a

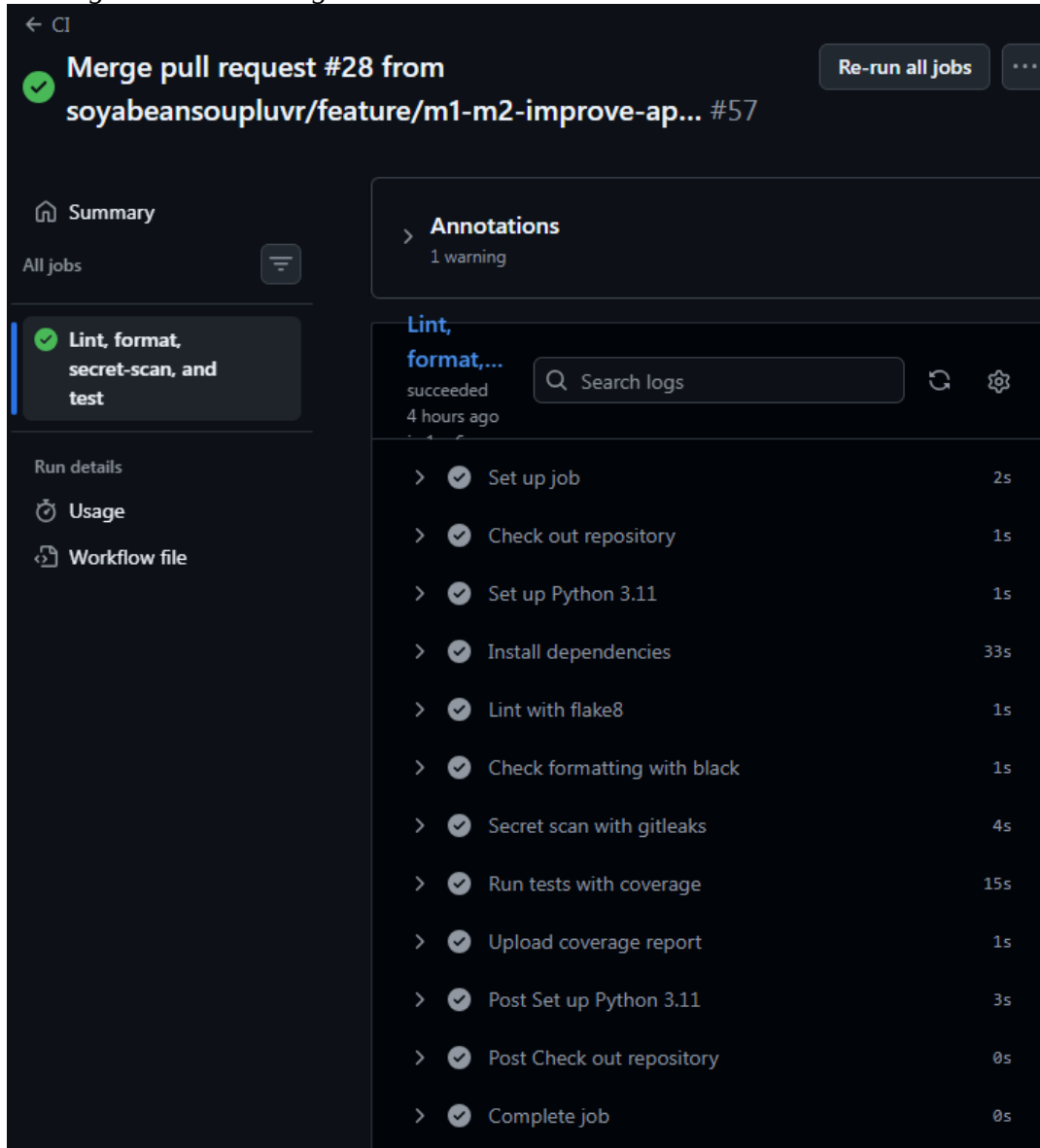
Note. Every authorization decision and protected store write is recorded as a chained event, so that an alteration of the history invalidates subsequent hashes.

Development evidence forms the second half of the output record. The continuous integration workflow runs on every push and pull request to the protected branches and executes style checking with flake8, formatting verification with black, secret scanning with gitleaks, and the full test suite with coverage reporting under a failure threshold. Release history is equally visible: the v0.1.0 tag records the ingestion, cleaning, protected storage, and audit foundation; the v0.9.0 tag records the tabbed interface, authentication, and end-to-end analytics wiring; and v1.0.0, published on July 31, 2026, alongside a Docker image and compose deployment, records the completed capstone release produced when develop merged into main. Figure 8 shows a passing workflow run, and the published releases are reproduced in Appendix A.



Figure 8

Passing Continuous Integration Run



Note. The run confirms that linting, formatting verification, secret scanning, and the coverage-gated test suite completed successfully before merge.

4.3 Evaluation Metrics

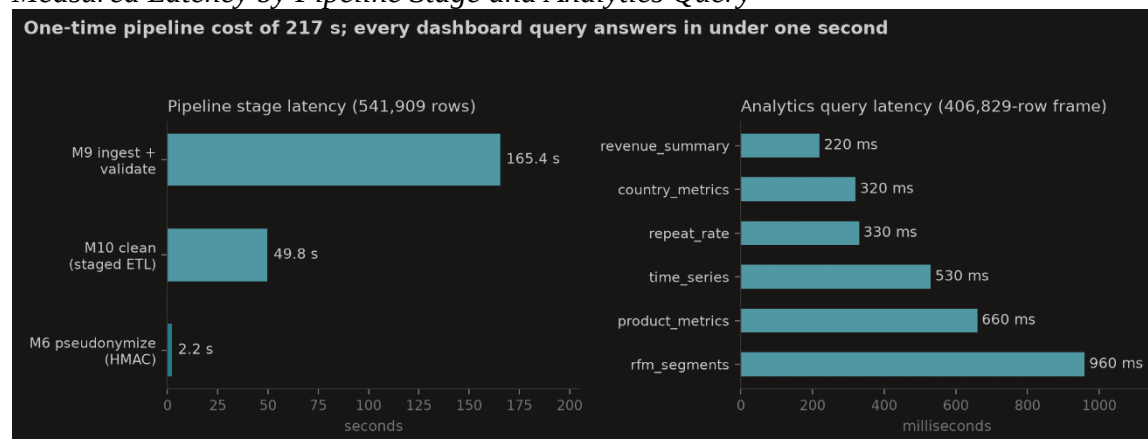
Evaluation combined quantitative measurement with a qualitative inspection of the interface, on the reasoning that numbers establish whether the system performs, and qualitative evidence establishes whether it is usable. Brereton (2023) argues that quantitative metrics answer how much and how many, while qualitative metrics answer why, and that decisions improve when

the two are read together. Wells (2025) similarly recommends a small balanced set of software quality metrics tied to concrete goals rather than an exhaustive dashboard of counters, and GeeksforGeeks (2025b) describes product, process, and project metrics as complementary views of quality. The metric set reported here follows that guidance: it covers pipeline performance, resource use, test adequacy, process gating, and interface quality, and it stops there.

Performance was measured on a sandbox profile with two virtual central processing units and 8 GB of memory using a timed benchmark script with memory tracing. Ingestion and validation of the workbook took 165.4 seconds, staged cleaning took 49.8 seconds, and pseudonymization of 406,829 rows took 2.2 seconds. The complete pipeline from ingestion through pseudonymization took 217.3 seconds, which corresponds to a throughput of 2,494 rows per second and a peak traced memory of 492.5 MiB. Interactive analytics are an order of magnitude faster because they read the cached snapshot: the revenue summary answered in 0.22 seconds, country metrics in 0.32 seconds, repeat rate in 0.33 seconds, the time series in 0.53 seconds, product metrics in 0.66 seconds, and recency, frequency, and monetary segmentation of 4,339 customers in 0.96 seconds. Every analytics query, therefore, completes in under one second. Figure 9 presents these latencies, and Table 3 consolidates the full metric set with its indicator type.

Figure 9

Measured Latency by Pipeline Stage and Analytics Query



Note. Bars show measured wall clock latency for the one-time protected pipeline stages and for each interactive analytics query. Workbook parsing dominates the 217.3-second bootstrap, while every dashboard query completes in under one second. Data from the project benchmark run on a two-processor, 8 GB sandbox.



Table 3

Consolidated Quantitative and Qualitative Evaluation Metrics

Metric	Value	Indicator type
End-to-end protected pipeline (ingest to pseudonymize)	217.3 s (one-time bootstrap)	Quantitative
Workbook ingest and validation (M9)	165.4 s	Quantitative
Staged cleaning (M10)	49.8 s	Quantitative
Pseudonymization of 406,829 rows (M6)	2.2 s	Quantitative
Fastest analytics query (revenue summary)	0.22 s	Quantitative
Slowest analytics query (segmentation, 4,339 customers)	0.96 s	Quantitative
Pipeline throughput	2,494 rows per second	Quantitative
Peak traced memory	492.5 MiB	Quantitative
Clean rows delivered to analytics	406,829 of 541,909 input rows	Quantitative
Unit tests passing	351 of 351	Quantitative
Branch-aware statement coverage	90.3 percent of 1,603 statements	Quantitative
Lowest module coverage (ingestion)	78.7 percent	Quantitative
Continuous integration coverage gate	60 percent floor enforced	Quantitative gate
Continuous integration quality gates	flake8, black, gitleaks, and pytest pass	Quantitative gate
Heuristic usability walkthrough	Status visibility, user vocabulary, and error prevention satisfied; bootstrap progress feedback flagged	Qualitative
Explainable chart captions	Formula, filters, exclusions, and row count shown on every figure	Qualitative
Role-appropriate disclosure	Tabs and analyses restricted by role; denials audited	Qualitative

Quality was measured through the test suite and the gates that enforce it. The suite contains 351 test functions distributed across files that mirror the source modules, with the largest concentrations in application wiring at 65 tests, analytics at 31, access control at 30, and the audit log at 27. All 351 tests pass under Python 3.12. Branch-aware coverage reaches 90.3 percent of 1,603 statements, with complete coverage of access control and pseudonymization, 97.8 percent of the logging module, 96.5 percent each of analytics, storage, and credentials, 91.7 percent of visualization, 88.7 percent of configuration, 86.0 percent of cleaning, and 78.7 percent of ingestion. Continuous integration enforces a 60 percent coverage floor, so the measured result has a wide margin above the gate, and the same workflow blocks merges when linting, formatting, or



secret scanning fails. Structural coverage of this kind is consistent with how practitioners actually design unit tests, which Runeson (2006) found to be predominantly informed by knowledge of internal control flow.

The qualitative evaluation was a heuristic walkthrough of the dashboard conducted against established usability principles rather than a controlled user study. Nielsen and Molich (1990) describe heuristic evaluation as a small set of inspectors judging an interface against general principles such as visibility of system status, match with the user's language, error prevention, consistency, and clear error recovery. Applying those principles produced three findings that the implementation satisfies. First, system status is visible because each chart states its formula, filters, exclusions, and row count, so a user is never asked to trust an unexplained number. Second, the interface speaks the language of the retail manager, using terms such as revenue, repeat purchase rate, top products, and customer segments rather than module names or column identifiers. Third, disclosure is role-appropriate: a viewer is never shown a control that will fail authorization, which prevents an error rather than reporting one. The walkthrough also identified one weakness: the first launch on a fresh deployment has a long bootstrap wait with limited progress feedback, which is discussed in Chapter 5 as a limitation rather than a defect.

4.4 System Testing and Validation

The integrated system was subjected to the full progression of testing levels recommended in the testing literature, from automated unit and integration checks through system testing on real data to acceptance scenarios expressed in the users' terms (Alex, 2023; Amazon Web Services, n.d.; GeeksforGeeks, 2026a, 2026b). Unit and integration testing re-ran the complete pytest suite on the development workstation: all 351 tests passed with branch-aware coverage of 90.3 percent of 1,603 statements, reproducing the Week 6 baseline and confirming that the coverage-gated continuous integration results are portable across environments. Figure 10 shows the verified run.

Figure 10

Verified Unit and Integration Test Run With Branch-Aware Coverage

```

>> py -m pytest --cov=src --cov-fail-under=60
..... [ 25%]
..... [ 50%]
..... [ 75%]
..... [100%]

----- coverage: platform win32, python 3.12.10-final-0 -----
Name                               Stmts   Miss Branch BrPart  Cover   Missing
-----
src\access.py                       54      0     18      0 100.0%
src\analytics.py                   233      5     54      5  96.5%  114, 561-578, 776, 873->872, 896
src\cleaning.py                    279     26     50     12  86.0%  110, 126-129, 149->exit, 272-280, 290, 293-294
src\config.py                      113     10     38      7  88.7%  141, 153, 163, 170, 174, 177, 219-222
src\credentials.py                  51      2      6      0  96.5%  48, 52
src\ingestion.py                   333     63     84     22  78.7%  209->exit, 215->exit, 239-240, 244, 250, 257,
  430-433, 449-458, 476-491, 509-512, 539->541, 542, 637, 639, 702-703
src\logs.py                        256      3     58      4  97.8%  56, 62->66, 158, 301
src\pseudonymize.py                28      0      6      0 100.0%
src\schema\__init__.py              0      0      0      0 100.0%
src\storage.py                     96      3     18      1  96.5%  288-289, 319
src\visualization.py               160      8     46      7  91.7%  503, 511, 532->535, 535->537, 572->574, 580, 5
-----
TOTAL                               1603    120    378     58  90.3%

Required test coverage of 60% reached. Total coverage: 90.31%
351 passed in 19.41s

```

Note. All 351 tests pass under Python 3.12.10, and coverage reaches 90.3 percent of 1,603 statements against the 60 percent continuous integration floor.

System testing then validated the entire pipeline end-to-end against its requirements on the genuine dataset, which is the defining characteristic of the level (BrowserStack, n.d.; Hrupa, 2024). A scripted run ingested the 541,909-row workbook through the M9 trust boundary in 212.7 seconds, cleaned it in five reported stages to 406,829 rows, pseudonymized the customer identifier column in 2.5 seconds, and wrote the encrypted snapshot, for a pipeline total of 264.0 seconds at 2,052 rows per second and a peak traced memory of 493.1 MiB. Every stage delta matched the values published in Section 4.2 exactly, demonstrating that the pipeline is deterministic and that the reported arithmetic is consistent run to run. Acceptance testing followed the three user roles through scripted scenarios designed as documented test cases with explicit expected results (Rana, 2023): an analyst read the protected customers view, a viewer was denied that view before any decryption occurred and the denial was audited, the same viewer read the aggregate revenue view, all six analytics queries answered in 1.03 seconds or less, and the hash-chained audit log verified

as intact at the end of the run. All fourteen system and acceptance cases passed, as shown in Figure 11.

Figure 11
System and Acceptance Test Run on the Full Dataset

```
SYSTEM TEST RUN
Started: 2026-08-02T14:47:25.250611+00:00
Source workbook: data/raw/Online Retail.xlsx (22.6 MiB)

=====
{"ts": "2026-08-02T14:47:25+00:00", "level": "INFO", "logger": "src.ingestion", "message": "ingest start source_id=91ec1df8b43a kind=xlsx"}
{"ts": "2026-08-02T14:50:57+00:00", "level": "INFO", "logger": "src.ingestion", "message": "ingest ok source_id=91ec1df8b43a rows=541909 accepted=541909 errors=0 warnings=0"}
[PASS] ST-01 ingestion accepts valid workbook: 541,909 rows ingested in 212.7 s
{"ts": "2026-08-02T14:51:09+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean stage=type_coercion in=541909 out=541909 details={'quantity_missing': 0, 'quantity_uncoerced': 0, 'unit_price_missing': 0, 'unit_price_uncoerced': 0, 'customer_id_missing': 135080, 'customer_id_uncoerced': 0, 'invoice_date_missing': 0, 'invoice_date_uncoerced': 0}"}
{"ts": "2026-08-02T14:51:09+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean stage=null_customer_drop in=541909 out=406829 details={'customer_id_rows_dropped': 135080}"}
{"ts": "2026-08-02T14:51:45+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean stage=cancellation_tagging in=406829 out=406829 details={'cancellations': 8905, 'adjustments': 0, 'returns': 8905, 'paired_cancellations': 2787, 'unpaired_cancellations': 6118, 'paired_returns': 2787, 'unpaired_returns': 6118, 'pairing_ineligible_returns': 0, 'pairing_ineligible_originals': 0, 'cancellations_without_negative_quantity': 0}"}
{"ts": "2026-08-02T14:51:45+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean stage=revenue_derivation in=406829 out=406829 details={'negative_revenue_rows': 8905, 'zero_revenue_rows': 40}"}
{"ts": "2026-08-02T14:51:45+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean stage=non_product_tagging in=406829 out=406829 details={'non_product_rows': 1786}"}
{"ts": "2026-08-02T14:51:45+00:00", "level": "INFO", "logger": "src.cleaning", "message": "clean done input=541909 output=406829 dropped=135080"}
  stage=type_coercion      in=541,909 out=541,909 dropped= 0
  stage=null_customer_drop in=541,909 out=406,829 dropped=135,080
  stage=cancellation_tagging in=406,829 out=406,829 dropped= 0
  stage=revenue_derivation in=406,829 out=406,829 dropped= 0
  stage=non_product_tagging in=406,829 out=406,829 dropped= 0
[PASS] ST-02 staged cleaning reports deltas: 541,909 -> 406,829 rows (135,080 dropped) in 48.1 s
[PASS] ST-03 pseudonymization leaves no raw identifiers: 406,829 rows pseudonymized in 2.54 s
{"ts": "2026-08-02T14:51:49+00:00", "level": "INFO", "logger": "src.logs", "message": "applied schema migration version=1 name=001_audit.sql"}
{"ts": "2026-08-02T14:51:49+00:00", "level": "INFO", "logger": "src.logs", "message": "applied schema migration version=2 name=002_users.sql"}
{"ts": "2026-08-02T14:51:49+00:00", "level": "INFO", "logger": "src.logs", "message": "applied schema migration version=3 name=003_seed_demo_users.sql"}
{"ts": "2026-08-02T14:51:49+00:00", "level": "INFO", "logger": "src.logs", "message": "audit append id=1 actor=pipeline action=protected_store_write outcome=success"}
[PASS] ST-04 encrypted snapshot written: snapshot 20260802T145148Z written in 0.7 s
  pipeline total 264.0 s, throughput 2,052 rows/s, peak memory 493.1 MiB
{"ts": "2026-08-02T14:51:49+00:00", "level": "INFO", "logger": "src.logs", "message": "audit append id=2 actor=analyst action=protected_store_read outcome=success"}
[PASS] AT-01 analyst reads customers view: 406,829 rows returned
{"ts": "2026-08-02T14:51:50+00:00", "level": "INFO", "logger": "src.logs", "message": "audit append id=3 actor=viewer action=access_denied outcome=failure"}
[PASS] AT-02 viewer denied customers view: denied: role 'viewer' is not permitted to read view 'customers'
{"ts": "2026-08-02T14:51:50+00:00", "level": "INFO", "logger": "src.logs", "message": "audit append id=4 actor=viewer action=protected_store_read outcome=success"}
[PASS] AT-03 viewer reads revenue view: 406,829 rows returned
{"ts": "2026-08-02T14:51:52+00:00", "level": "INFO", "logger": "src.logs", "message": "audit append id=5 actor=analyst action=protected_store_read outcome=success"}
[PASS] AT-04 revenue summary: answered in 0.18 s
[PASS] AT-05 product metrics: answered in 0.66 s
[PASS] AT-06 time series: answered in 0.49 s
[PASS] AT-07 country metrics: answered in 0.31 s
[PASS] AT-08 repeat rate: answered in 0.33 s
[PASS] AT-09 RFM segmentation: answered in 1.03 s
[PASS] AT-10 audit chain verification: hash chain intact

=====
RESULT: 14/14 system and acceptance cases passed
=====
```

Note. The staged cleaning deltas reconcile 541,909 input rows to 406,829 clean rows, and all fourteen system and acceptance cases pass, including the audited denial and the audit chain verification.

Testing during the term also found and resolved three defects worth recording. Pandas silently promoted customer identifiers to floating-point values, which would have changed the exact bytes that HMAC-SHA256 receives and produced unstable pseudonyms; the fix normalizes



the identifier to a string at ingestion. Malformed database configuration originally failed deep inside the storage layer; validation moved into the immutable settings object so that bad configuration fails at construction with a clear error. A realistic 64-character hexadecimal test fixture triggered the secret scanner in continuous integration, so the fixture was rewritten to use low-entropy bytes rather than teaching the scanner to ignore it. The remaining documented weaknesses are the 78.7 percent coverage of the ingestion module, whose uncovered branches are concentrated in rejection paths that require awkward binary fixtures, and the limited progress feedback during the first-launch bootstrap.



DISCUSSION

5.1 Interpretation of Results

The most important pattern in the measurements is the separation between a single expensive preparation cost and a uniformly cheap interaction cost. The 217.3-second pipeline is paid once, at bootstrap, because the idempotent snapshot writer refuses to repeat work that has already been done, and every subsequent user action reads the encrypted snapshot instead. Read against the sub-second analytics latencies, this means the system behaves like an analytical product rather than a batch script: a manager who opens the dashboard is not waiting on ingestion, cleaning, or hashing, but on a query that returns in under a second. The design consequence is that optimization effort belongs in the bootstrap path, and the operational consequence is that a deployment plan should warm the snapshot before users arrive rather than letting the first visitor absorb the cost. The system run strengthened this interpretation, because a second full execution of the pipeline on the development workstation reproduced both the stage arithmetic exactly, with identical row deltas and a 264.0-second total that reflects the different hardware profile, which is the behavior a deterministic pipeline should exhibit.

Within the bootstrap, the cost distribution is lopsided in an instructive way. Workbook parsing accounts for 165.4 of the 217.3 seconds, roughly three-quarters of the total, while staged cleaning accounts for 49.8 seconds and pseudonymization for only 2.2 seconds. The privacy mechanism that a reader might expect to be expensive is, in fact, the cheapest stage measured, because keyed HMAC-SHA256 over a single column is a linear pass over 406,829 values. This is a practically useful finding for small-business analytics: pseudonymizing identifiers before analysis costs about one percent of the preparation budget, so privacy protection is not a performance argument against protected pipelines. The genuine bottleneck is the input format. Spreadsheet parsing carries interpretive overhead that a columnar or delimited format does not, which points directly at the ingestion improvement proposed later in this chapter.

Memory behavior confirms that the chosen deployment target is adequate. Peak traced allocation was 492.5 MiB against 8 GB available, which leaves the full dataset comfortably inside



a small cloud instance and, more importantly, inside a single process. The throughput figure of 2,494 rows per second should be read as an end-to-end figure that includes validation, staged cleaning, and cryptographic work, not as a raw parsing rate, and it is therefore a conservative planning number for datasets of similar shape.

The test results support a similar reading of depth against breadth. Overall branch-aware coverage of 90.3 percent across 1,603 statements, with 351 passing tests, indicates that the modules that carry the project's guarantees are thoroughly exercised: access control and pseudonymization are fully covered, and logging, storage, analytics, and credentials all exceed 96 percent. The weakest module is ingestion at 78.7 percent, and that gap is meaningful rather than cosmetic, because ingestion is the module that faces untrusted input. The explanation is structural: many ingestion branches depend on malformed binary workbooks, media type mismatches, and oversized files that are awkward to construct as fixtures, so the uncovered lines are concentrated in rejection paths rather than in the accept path used every run. The interpretation is not that the trust boundary is untested, since thirteen ingestion tests assert documented accept and reject behavior, but that the residual risk in the system is concentrated exactly where coverage is lowest, which is a useful thing to know and to schedule.

5.2 Comparison with Literature Findings

The project's configuration management practice aligns closely with the goals that Summers (2020) sets for software configuration management: identifying the items under control, controlling how they change, recording their status, and producing an integration build and release that can be reproduced. The repository realizes those goals concretely. Source modules, tests, the workflow definition, the dependency manifests, and the environment template are all versioned items; change control is enforced by protected branches that require pull requests and passing status checks; status accounting is visible in more than thirty merged pull requests, conventional commit messages that name the module scope, and annotated tags; and the integration build is the continuous integration workflow whose green run is the precondition for merge.

Estublier (2000) argues that configuration management matured from file versioning into process support, covering build, deployment, and coordination as well as history. The project reflects that broader definition, because the same repository that stores history also stores the workflow that gates quality, the environment template that defines configuration, and the release



notes that communicate state. Bennett (2026) describes the standard elements of configuration management as identification, control, status accounting, and audit, and the hash-chained audit log extends that last element from the development process into the running system: the product audits its own access decisions with the same discipline the repository applies to its own changes. SNS Institutions (n.d.) presents configuration management as the discipline that keeps a team's baselines consistent as change accumulates, which is the reason a single-developer project still benefits from pull requests, since the reviewable unit, rather than the reviewer, is what preserves the baseline.

Version control literature explains why the tagging strategy matters. Zolkifli et al. (2018) review version control systems as mechanisms for recording change history and coordinating parallel work, and note that their value grows with the traceability they provide. In this project, traceability is the point: any figure on the dashboard can be traced to a release, a merge commit, a pull request, and an individual commit, allowing an inspector or a future maintainer to reproduce a result rather than accept it. Runeson (2006) found that practitioner unit testing is predominantly structural, designed with knowledge of internal control flow, and the suite here matches that finding, since branch coverage targets such as denied reads and rejected schemas were reached deliberately, with specification-based cases added at the ingestion boundary, where documented behavior is the contract.

The evaluation approach follows the measurement literature rather than improvising. Brereton (2023) recommends combining qualitative and quantitative indicators, which is why Table 3 reports latency, throughput, memory, test count, and coverage alongside heuristic usability findings. Wells (2025) and GeeksforGeeks (2025b) both caution that metrics are only useful when tied to decisions and thresholds, and the project honors that by gating merges on the coverage floor and by treating the ingestion coverage figure as an action item rather than a statistic. Nielsen and Molich (1990) supply the inspection method used in the absence of end users, and their own caution that heuristic evaluation is a discount technique rather than a substitute for user testing is reflected in the limitations below.

Deployment practice also has supporting literature. Sumo Logic (n.d.) describes software deployment as the full set of activities that make a build available for use, which reframes the environment template, the setup commands, and the container image as parts of the product rather



than as documentation afterthoughts. ByteByteGo (2023) surveys common deployment strategies, including recreate, rolling, blue-green, and canary releases. The single-process nature of this prototype currently limits it to a recreate-style rollout with rollback via image tag, which is an honest constraint rather than a preference. Brown (2026) argues that documentation succeeds when it is task-oriented, current, and located where the reader already is. The project follows that guidance by keeping setup steps, environment variable descriptions, and verification commands in the repository README alongside the code they describe.

5.3 Implications and Limitations

The practical implication of the results is that a small business can obtain privacy-preserving analytics without specialized infrastructure. A pipeline that pseudonymizes before storage, encrypts at rest, authorizes before decryption, and audits every decision ran comfortably on two virtual processors and used under half a gigabyte of memory, while answering every interactive query in under a second. That combination suggests the protective pattern is affordable at the scale most small retailers operate at, and that the usual trade-off between privacy and responsiveness does not apply at this data volume.

The limitations are real and bounded. The evaluation used a single historical dataset of 541,909 rows, so behavior on streaming data, on larger volumes, or on differently shaped schemas remains unmeasured. Persistence relies on SQLite for the audit log and on encrypted files for snapshots, neither of which supports concurrent multi-node access, so the current design is a single-instance design by construction. The Streamlit runtime is a single process serving the interactive session, which bounds concurrency and makes the recreate-style deployment noted above the only straightforward rollout. The usability findings come from a heuristic, expert-style inspection rather than from observing real managers, so they identify likely problems without measuring task success, satisfaction, or error rates. The benchmark reflects one hardware profile, and although the Week 7 run reproduced the original figures closely, the reported numbers should be treated as representative rather than as a distribution. Finally, the ingestion cost of the spreadsheet format is a limitation of the current input contract, and the 78.7 percent coverage of that module reflects the residual risk that remains after the trust boundary was hardened.



5.4 Future Work

Four improvements follow directly from the measurements. The first targets the dominant cost: adding a delimited fast path and chunked or streaming ingestion would let the pipeline avoid whole-workbook parsing, which currently consumes 165.4 of the 217.3-second bootstrap, and would also reduce peak memory by processing the input in bounded pieces. The second addresses persistence: moving the audit log to a managed relational service and snapshots to object storage would replace the single-node assumptions of SQLite and local files, allowing several application instances to share one protected store while retaining the authorize-before-decrypt ordering. The third addresses operations: placing the container behind an orchestrator would enable rolling or blue-green rollouts rather than the recreate pattern currently available, removing the visible restart from the update path (ByteByteGo, 2023).

The fourth improvement concerns evidence about people and about model behavior. The heuristic walkthrough should be followed by a small moderated study with actual retail managers, measuring task completion, time on task, and comprehension of the explained chart captions, since Nielsen and Molich (1990) present inspection as a precursor to, not a replacement for, user testing. Segmentation should move from a one-time fit to a scheduled retraining job with drift monitoring, so that changes in the adjusted Rand index stability check or in segment sizes raise an alert instead of silently altering the meaning of a segment label. Supporting both, the protected pipeline should be packaged as a command line entry point that a scheduler can invoke, allowing snapshots to be refreshed on a schedule, warmed before users arrive, and verified in continuous integration as a first-class command rather than as an implicit side effect of application startup.

Post-deployment life beyond these improvements will follow the standard maintenance taxonomy of corrective, adaptive, perfective, and preventive work (5-Minute Lessons by Victor, 2024; GeeksforGeeks, 2025c; Thales, n.d.). The Week 7 maintenance plan submitted alongside this report assigns each anticipated activity to one of those types, sets a monitoring and update cadence, and pairs the likely post-deployment risks, including dependency vulnerabilities, schema drift, and the bootstrap wait, with specific mitigation and contingency measures.



5.5 Deployment Approach and Configuration Summary

The selected deployment approach is a containerized image running on a small cloud virtual machine, and the v1.0.0 release ships the Dockerfile and compose definition that implement it. Two alternatives were considered. A managed platform such as Streamlit Community Cloud would have been the simplest option, requiring almost no operational work, but it offers less control over where retail data rests and how secrets are handled, which conflicts with a project whose subject is protected analytics. A purely on-premises deployment would have offered maximum data control, but it would have placed the full operational burden of patching, backup, and availability on a single developer. The container on a cloud instance sits between those poles: the image pins the Python runtime and every dependency, so a build is reproducible; configuration is injected as environment variables rather than baked into the artifact, so the same image serves development and production; and rollback is a matter of running the previous image tag. Sumo Logic (n.d.) treats exactly this bundle of build, configure, and release activities as the deployment process.

Configuration is documented so that a reviewer can reproduce the environment. The prerequisites are Python 3.12 or newer, pip, and git on Linux, macOS, or Windows, plus the roughly 23 MB dataset placed in the raw data directory. Runtime dependencies are pinned in the requirements manifest and include Streamlit 1.36.0, streamlit-authenticator 0.3.2, pandas 2.2.2, numpy 1.26.4, openpyxl 3.1.5, pyarrow 16.1.0, scikit-learn 1.5.0, SQLAlchemy 2.0.31, plotly 5.22.0, python-dotenv 1.0.1, and cryptography 42.0.8, while development dependencies pin pytest 8.2.2, pytest-cov 5.0.0, flake8 7.1.0, and black 24.4.2. Environment variables control the server port and address, the raw and processed data directories, the SQLite audit database URL, the small-group suppression threshold of five, a request rate limit of 120 requests per minute, the segmentation holdout fraction of one tenth, the stability threshold of 0.7, and the log level, format, and directory. The one secret, the pseudonymization key, is never committed: it must be at least 32 hexadecimal characters, is generated locally with a single secrets module command, is supplied through the environment, and is rotated quarterly. Rotation changes every pseudonym, so a rotation is deliberately treated as a snapshot rebuild rather than as a transparent operation. The setup and verification command sequence is reproduced in Appendix B.



Configuration management is what makes that sequence trustworthy rather than merely convenient. Because main and develop are protected and accept only reviewed pull requests that pass checks, the code that reaches a release has already been linted, formatted, scanned for secrets, and tested. Because milestones are annotated tags with published releases, covering v0.1.0 for ingestion, cleaning, protected storage, and audit foundation; v0.9.0 for the end-to-end wired interface; and v1.0.0 for the completed capstone with its container deployment, a deployer can build an image from an immutable pointer rather than a moving branch. Summers (2020) describes this as the integration, build, and release function of configuration management; Bennett (2026) as the status accounting and audit elements; and Estublier (2000) as the extension of configuration management into process and deployment support. Brown (2026) completes the picture by insisting that the instructions live with the artifact, which is why the environment template and the verification command are versioned beside the code they configure.



CONCLUSION

6.1 Summary of Findings

The capstone delivered a working interactive sales analytics dashboard whose numbers can be explained and whose data path can be audited. On the real UCI Online Retail dataset, the system accepted 541,909 transaction rows through a validating trust boundary, reduced them to 406,829 clean rows with every removal reported as a stage delta, pseudonymized customer identifiers with keyed HMAC-SHA256, and served role-gated analytics from a Fernet-encrypted snapshot. Preparation cost 217.3 seconds once, at a throughput of 2,494 rows per second and a peak of 492.5 MiB, while every interactive query returned in under one second. Quality evidence matched the functional result: 351 tests pass with 90.3 percent branch-aware coverage of 1,603 statements, and continuous integration blocks any merge that fails linting, formatting, secret scanning, or the coverage floor. The system and acceptance testing then re-executed the full pipeline on the same dataset and reproduced the published arithmetic exactly, with all fourteen scenario cases passing, including the audited denial of an unauthorized read and the verification of the hash-chained audit history. A heuristic usability walkthrough found the interface consistent with recognized principles in its visibility of system status, its use of the manager's vocabulary, and its role-appropriate disclosure, with the bootstrap wait as the notable rough edge.

Those findings imply that privacy protection and analytic responsiveness were not in tension at this scale. The cryptographic stage that most directly implements the privacy commitment accounted for 2.2 seconds of a 217-second pipeline, which is close to negligible, while the interpretive cost of the spreadsheet input format accounted for three-quarters of it. For a small business considering protected analytics, the honest advice from this evaluation is to focus on data formats and ingestion design first, and to treat pseudonymization, encryption at rest, authorization, and audit logging as affordable defaults rather than premium features.

6.2 Contributions

The project contributes three transferable patterns rather than a single application. The first is an ordering discipline for protected analytics that can be stated in one line: pseudonymize,



then encrypt, then authorize before decryption, and audit every decision in a hash-chained log. Ordering the steps this way means that a denied request never materializes plaintext, that no analytic module ever holds a natural identity key, and that the history of access is tamper-evident. Small organizations that cannot afford a data platform team can adopt this approach within a single process and still obtain the guarantees usually associated with larger architectures.

The second contribution is the explainable chart contract. Every figure returned by the visualization layer includes the formula that produced it, the filters and exclusions applied, and the number of rows behind it, so provenance travels with the visual rather than living in a separate document. This turns explainability into a structural property of the interface and treats usability and correctness as the same problem, since a caption that cannot be generated signals that the underlying metric is underspecified.

The third contribution is an evidence-first development workflow suited to a single developer who must nonetheless prove process discipline. Protected branches, pull requests, conventional commits scoped to numbered modules, annotated tags with published releases, and a continuous integration workflow that lints, formats, scans for secrets, and tests with a coverage gate together convert habits into artifacts. The practical result is traceability in both directions: a reviewer can move from a dashboard figure back to a release, a pull request, and a commit, or forward from a commit to the automated evidence that admitted it.

6.3 Lessons Learned and Next Improvements

Three challenges taught more than the successes did. The first was a type coercion problem: pandas repeatedly promoted customer identifiers to floating point values, which changed the exact bytes that HMAC-SHA256 requires and would have produced unstable pseudonyms. The fix was to normalize the identifier to a string at ingestion and to drop rows that cannot satisfy the pseudonymization contract, which is why the cleaning stage reports 135,080 removals rather than hiding them. The lesson is that cryptographic operations impose data typing requirements upstream of themselves, and that those requirements belong in the ingestion contract rather than in the module that discovers the problem.

The second lesson concerned ownership of validation. Malformed database configuration originally failed deep within the storage layer, making the resulting error hard to diagnose. Moving validation into an immutable settings object made bad configuration fail at construction with a



clear configuration error, and a previously passing test that expected a database error then failed. That failure was informative rather than annoying, because it documented that an invariant had changed owners; rewriting the test to expect the new error made the boundary explicit. Testing also reshaped the cleaning rules, forcing an unambiguous decision on rows without usable customer identifiers, and hardened the analytics input contract so that a malformed frame fails immediately rather than producing plausible but misleading metrics.

The third lesson came from the secret scanner. A realistic 64-character hexadecimal test fixture triggered a leaked credential warning in continuous integration. Suppressing the warning would have been faster, but the fixture was rewritten to use low-entropy bytes, so the test still exercises key validation without resembling a real key. The lesson generalizes: when an automated gate objects, the cheapest response is usually to remove the ambiguity it detected rather than to teach it to ignore it. A related friction appeared in branch protection, where synchronizing main and develop after a release required deliberate pull requests rather than a convenient direct push, which is precisely the cost that makes the history trustworthy.

The next improvements follow the limitations rather than the ambitions. Ingestion should gain a delimited fast path and chunked processing to address the dominant cost and simultaneously improve the weakest coverage figure. Persistence should move to a managed database and object storage so the protected store can serve multiple instances. Deployment should move behind an orchestrator so that updates roll rather than recreate. Segmentation should be retrained on a schedule with drift alerts on the stability check. Above all, the heuristic usability findings should be replaced by observations of real retail managers completing real tasks, because the one claim this evaluation cannot yet make quantitatively is that the dashboard helps its users make better decisions.



REFERENCES

- 5-Minute Lessons by Victor. (2024, June 28). *Types of software maintenance (corrective, adaptive, perfective, and preventive)* [Video]. YouTube.
https://www.youtube.com/watch?v=_8aNZZaaoSQ
- A Dev' Story. (2020, September 27). *Getting the basics—Software architecture introduction (Part 1)* [Video]. YouTube. <https://www.youtube.com/watch?v=8UuLgOf20Ho>
- Alex, H. (2023, March 3). *5 types of testing software every developer needs to know!* [Video]. YouTube. <https://www.youtube.com/watch?v=YaXJeUkBe4Y>
- Amazon Web Services. (n.d.). *What is unit testing?* <https://aws.amazon.com/what-is/unit-testing/>
- Bennett, L. (2026, July 9). *Software configuration management in software engineering*. Guru99.
<https://www.guru99.com/software-configuration-management-tutorial.html>
- Bergmann, F. (2021, March 17). *My favorite open source project management tools*. Opensource.com. <https://opensource.com/article/21/3/open-source-project-management>
- Bit.ai Editorial Team. (n.d.). *Create a software design document easily: Steps & template inside*. Bit.ai Blog. <https://blog.bit.ai/software-design-document/>
- Brereton, J. (2023, November 27). *Qualitative vs. quantitative metrics: A comprehensive comparison*. LaunchNotes. <https://www.launchnotes.com/blog/qualitative-vs-quantitative-metrics-a-comprehensive-comparison>
- BrowserStack. (n.d.). *What is system testing? (2026 comprehensive guide)*. <https://www.browserstack.com/guide/what-is-system-testing>
- Brown, J. (2026, April 21). *Software documentation guide: Definition, examples, downloadable templates + how to write*. Helpjuice. <https://helpjuice.com/blog/software-documentation>
- ByteByteGo. (2023, June 6). *Top 5 most-used deployment strategies* [Video]. YouTube.
<https://www.youtube.com/watch?v=AWVTKBUnoIg>
- ByteMonk. (2024, October 11). *Top 9 software architecture patterns every developer must know!* [Video]. YouTube. <https://www.youtube.com/watch?v=126ALse1rWA>



- Chen, D. (2015). *Online retail* [Data set]. UCI Machine Learning Repository.
<https://doi.org/10.24432/C5BW33>
- Chen, H., Chiang, R. H. L., & Storey, V. C. (2012). Business intelligence and analytics: From big data to big impact. *MIS Quarterly*, 36(4), 1165–1188.
<https://doi.org/10.2307/41703503>
- Corbin. (2025, March 13). *How to use GitHub for beginners* [Video]. YouTube.
<https://www.youtube.com/watch?v=a9u2yZvsqHA>
- Davies, S. (2024). *Blueprints: Creating, describing, and implementing designs for larger-scale software projects* (Version 2.5). University of Mary Washington.
<https://open.umn.edu/opentextbooks/textbooks/blueprints-creating-describing-and-implementing-designs-for-larger-scale-software-projects-davis>
- Estublier, J. (2000). Software configuration management: A roadmap. In *Proceedings of the Conference on the Future of Software Engineering* (pp. 279–289). ACM.
<https://doi.org/10.1145/336512.336576>
- European Parliament and Council of the European Union. (2016). *Regulation (EU) 2016/679 (General Data Protection Regulation)*. *Official Journal of the European Union*.
<https://eur-lex.europa.eu/eli/reg/2016/679/oj>
- Flyvbjerg, B. (2008). Curbing optimism bias and strategic misrepresentation in planning: Reference class forecasting in practice. *European Planning Studies*, 16(1), 3–21.
<https://doi.org/10.1080/09654310701747936>
- GeeksforGeeks. (2025a, July 11). *Prototyping model—Software engineering*.
<https://www.geeksforgeeks.org/software-engineering/software-engineering-prototyping-model/>
- GeeksforGeeks. (2025b, July 15). *Measuring software quality using quality metrics*.
<https://www.geeksforgeeks.org/software-engineering/measuring-software-quality-using-quality-metrics/>
- GeeksforGeeks. (2025c, September 29). *Software maintenance—Software engineering*.
<https://www.geeksforgeeks.org/software-engineering/software-engineering-software-maintenance/>



- GeeksforGeeks. (2026a, June 24). *Unit testing*. <https://www.geeksforgeeks.org/software-testing/unit-testing-software-testing/>
- GeeksforGeeks. (2026b, July 6). *System testing—Software engineering*. <https://www.geeksforgeeks.org/software-testing/system-testing/>
- Hrupa, A. (2024, March 19). *What is system testing in software testing?* TestFort. <https://testfort.com/blog/what-is-system-testing-in-software-testing>
- John, J. M., Shobayo, O., & Ogunleye, B. (2023). An exploration of clustering algorithms for customer segmentation in the UK retail market. *Analytics*, 2(4), 809–823. <https://doi.org/10.3390/analytics2040042>
- Knowledge Academy. (2023, April 1). *What is Azure DevOps | Azure DevOps for beginners | Azure DevOps explained* [Video]. YouTube. https://www.youtube.com/watch?v=ZCROe1Wu_-k
- Krüger, G. (2025, July 11). *How to write a software requirements specification (SRS) document*. Perforce. <https://www.perforce.com/blog/alm/how-write-software-requirements-specification-srs-document>
- Laptick, S. (2022, November 9). *How SDLC prototype model helps build comprehensive apps when requirements are unclear*. XB Software. <https://xbsoftware.com/blog/prototype-model-sdlc/>
- Letaw, L. (2024). *Handbook of software engineering methods*. Oregon State University. <https://open.oregonstate.edu/setextbook/>
- Lucid Software. (n.d.). *How to create software design documents*. <https://lucid.co/blog/how-to-create-software-design-documents>
- Mehrabi, N., Morstatter, F., Saxena, N., Lerman, K., & Galstyan, A. (2021). A survey on bias and fairness in machine learning. *ACM Computing Surveys*, 54(6), 1–35. <https://doi.org/10.1145/3457607>
- Miller, R. (2026, May 22). *Capstone project outline: Structure & examples*. DoMyEssay. <https://domyessay.com/blog/capstone-project-outline>
- Narasimhan, A., Sham, R., & Subramanian, H. (2023). *A beginner's handbook to DevOps* [White paper]. Dell Technologies.



- https://learning.dell.com/content/dam/dell-emc/documents/en-us/2023KS_Narasimhan-A_Beginner%27s_Handbook_to_DevOps.pdf
- National Institute of Standards and Technology. (2020). *NIST privacy framework: A tool for improving privacy through enterprise risk management, version 1.0*.
<https://doi.org/10.6028/NIST.CSWP.01162020>
- Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems* (pp. 249–256). ACM.
<https://doi.org/10.1145/97243.97281>
- NxtChair. (2024, September 19). *How to conduct a feasibility study—Step-by-step guide for projects | Types of feasibility study* [Video]. YouTube. <https://www.youtube.com/watch?v=hiXcrGJQpi8>
- Oguz, A. (2025). *Project management* (2nd ed.). MSL Academic Endeavors.
<https://pressbooks.ulib.csuohio.edu/projectmanagement2ndedition/>
- OWASP Foundation. (2025). *OWASP Top 10:2025*. <https://owasp.org/Top10/2025/>
- Paperpile LLC. (n.d.). *How to write a problem statement*. <https://paperpile.com/g/how-to-write-a-problem-statement/>
- Parzych, D. (2019, December 30). *8 must-read DevOps articles for success in 2020*. Opensource.com. <https://opensource.com/article/19/12/devops-resources>
- PhD Assistance. (2020, December 14). *How to identify the appropriate PhD dissertation research problem: Identification in computer science research and ways to evaluate it*. <https://www.phdassistance.com/blog/how-to-identify-the-appropriate-phd-dissertation-research-problem-identification-in-computer-science-research-explain-the-ways-to-evaluate-it/>
- Rana, K. (2023, September 23). *What is a test case? Test case examples*. ArtOfTesting.
<https://artoftesting.com/test-case>
- Red Hat. (2025, April 29). *What is DevOps?* <https://www.redhat.com/en/topics/devops/what-is-devops>
- Research with Dr. Saeed. (2023, June 14). *How to write a problem statement for your research: A step-by-step guide* [Video]. YouTube. <https://www.youtube.com/watch?v=iaGpj8ViH1w>



- Runeson, P. (2006). A survey of unit testing practices. *IEEE Software*, 23(4), 22–29.
<https://doi.org/10.1109/MS.2006.91>
- Sheppard, V. A. (2019). *An introduction to research methods in sociology*. Justice Institute of British Columbia. <https://pressbooks.bccampus.ca/researchmethods/>
- Simplilearn. (2021, February 16). *DevOps in 5 minutes | What is DevOps? | DevOps tutorial for beginners* [Video]. YouTube. <https://www.youtube.com/watch?v=Xrgk023l4II>
- SNS Institutions. (n.d.). *Software configuration management | OOSE | SNS Institutions* [Video]. YouTube. <https://www.youtube.com/watch?v=wBjo-g2bdOo>
- Souppaya, M., Scarfone, K., & Dodson, D. (2022). *Secure software development framework (SSDF) version 1.1* (NIST Special Publication 800-218). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-218>
- Summers, B. L. (2020). *Effective methods for software engineering*. Auerbach Publications.
<https://doi.org/10.1201/9781003025665>
- Sumo Logic. (n.d.). *What is software deployment?*
<https://www.sumologic.com/glossary/software-development/>
- SupersimpleDev. (2021, June 13). *Git branching and merging—Detailed tutorial* [Video]. YouTube. <https://www.youtube.com/watch?v=Q1kHG842HoI>
- Swiftful Thinking. (2024, May 16). *Adding tags, versioning, and releases in GitHub | Git & source control #13* [Video]. YouTube. <https://www.youtube.com/watch?v=D3W66tfYGuc>
- Tech With Diego. (2025, August 23). *Git and GitHub explained in 7 minutes | Git and GitHub tutorial* [Video]. YouTube. <https://www.youtube.com/watch?v=y-an0v208A0>
- Thales. (n.d.). *4 types of software maintenance & what is software maintenance?*
<https://cpl.thalesgroup.com/software-monetization/four-types-of-software-maintenance>
- Tsiu, S. V., Ngobeni, M., Mathabela, L., & Thango, B. A. (2025). Applications and competitive advantages of data mining and business intelligence in SMEs performance: A systematic review. *Businesses*, 5(2), Article 22. <https://doi.org/10.3390/businesses5020022>
- Walker, V. (2022, March 16). *14 software architecture design patterns to know*. Red Hat.
<https://www.redhat.com/en/blog/14-software-architecture-patterns>



Wells, J. (2025, April 16). *Top 12 software quality metrics to measure and why*. LinearB.

<https://linearb.io/blog/software-quality-metrics>

Yigitbasioglu, O. M., & Velcu, O. (2012). A review of dashboards in performance management: Implications for design and research. *International Journal of Accounting Information Systems*, 13(1), 41–59. <https://doi.org/10.1016/j.accinf.2011.08.002>

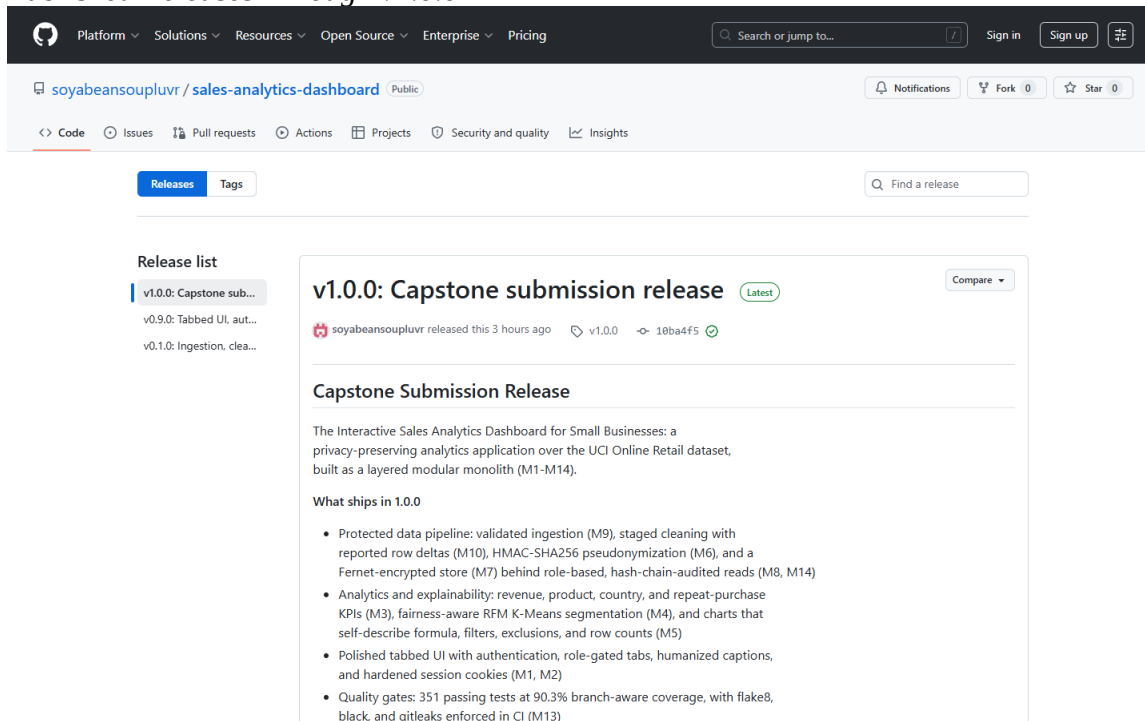
Zolkifli, N. N., Ngah, A., & Deraman, A. (2018). Version control system: A review. *Procedia Computer Science*, 135, 408–415. <https://doi.org/10.1016/j.procs.2018.08.191>

APPENDIX A

Repository, Release, and Continuous Integration Evidence

The figures in this appendix reproduce the development evidence referenced in Chapters 4 and 5. Figure A1 shows the published release list, including the v1.0.0 capstone submission release, as of July 31, 2026. Figures A2 and A3 show the annotated milestone releases that preceded it, and Figure A4 shows the continuous integration history through the release merge.

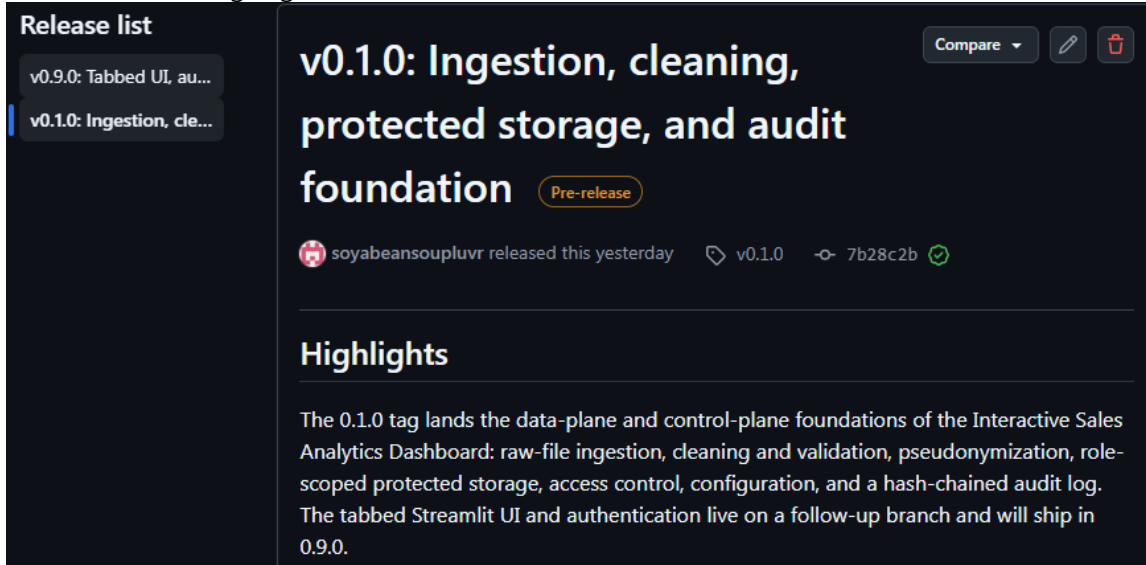
Figure A1
Published Releases Through v1.0.0



Note. The v1.0.0 capstone submission release ships the protected pipeline, the explainable analytics, the tabbed authenticated interface, and the quality gates, alongside the v0.1.0 and v0.9.0 milestones.

Figure A2

v0.1.0 Release Highlights



Release list

- v0.9.0: Tabbed UI, au...
- v0.1.0: Ingestion, cle...**

v0.1.0: Ingestion, cleaning, protected storage, and audit foundation Pre-release

soyabeansoupluvr released this yesterday v0.1.0 7b28c2b

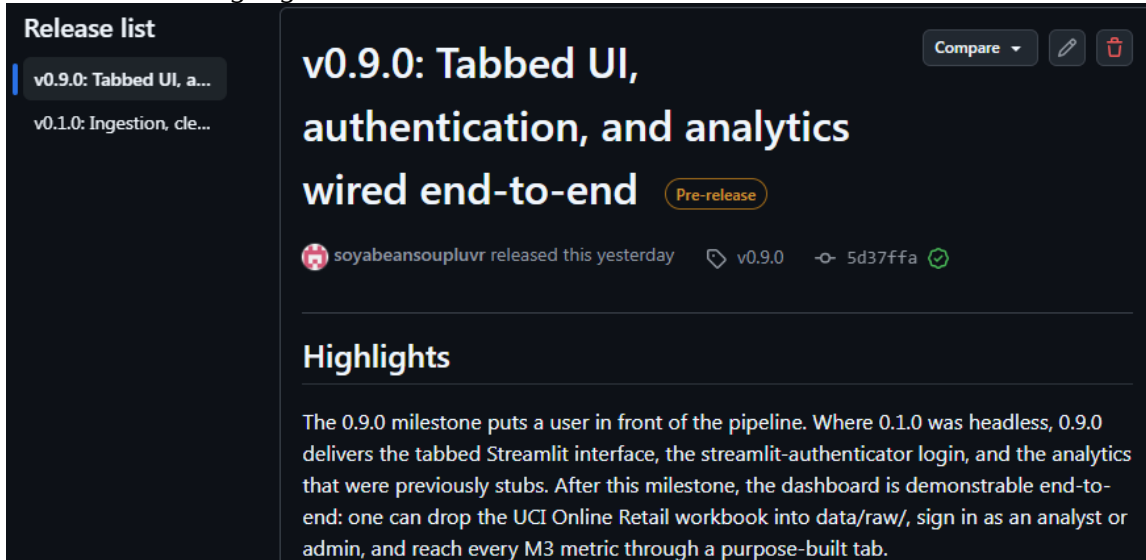
Highlights

The 0.1.0 tag lands the data-plane and control-plane foundations of the Interactive Sales Analytics Dashboard: raw-file ingestion, cleaning and validation, pseudonymization, role-scoped protected storage, access control, configuration, and a hash-chained audit log. The tabbed Streamlit UI and authentication live on a follow-up branch and will ship in 0.9.0.

Note. The v0.1.0 annotated tag records the ingestion, cleaning, protected storage, and audit foundation.

Figure A3

v0.9.0 Release Highlights



Release list

- v0.9.0: Tabbed UI, a...**
- v0.1.0: Ingestion, cle...

v0.9.0: Tabbed UI, authentication, and analytics wired end-to-end Pre-release

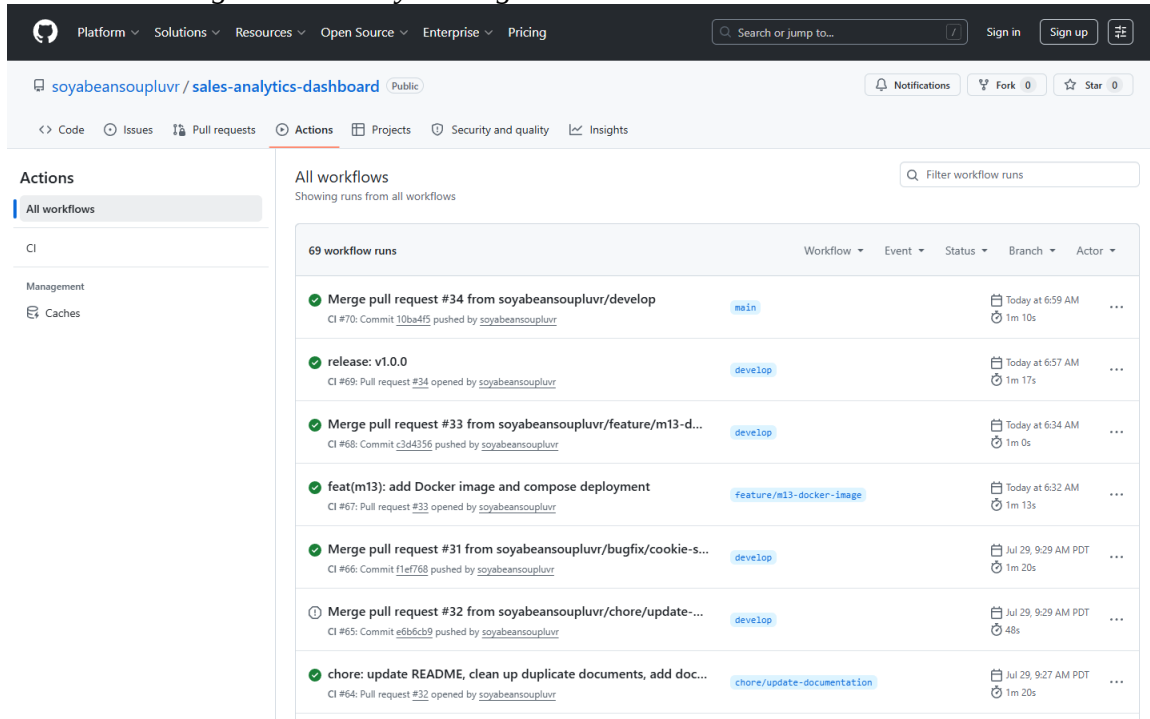
soyabeansoupluvr released this yesterday v0.9.0 5d37ffa

Highlights

The 0.9.0 milestone puts a user in front of the pipeline. Where 0.1.0 was headless, 0.9.0 delivers the tabbed Streamlit interface, the streamlit-authenticator login, and the analytics that were previously stubs. After this milestone, the dashboard is demonstrable end-to-end: one can drop the UCI Online Retail workbook into data/raw/, sign in as an analyst or admin, and reach every M3 metric through a purpose-built tab.

Note. The v0.9.0 annotated tag records the tabbed interface, authentication, and end-to-end analytics wiring.

Figure A4
Continuous Integration History Through the v1.0.0 Release



Note. Run 70 admits the v1.0.0 capstone release to main after linting, formatting, secret scanning, and the coverage-gated test suite pass.



APPENDIX B

Setup and Verification Commands

The following command sequence reproduces the environment on Linux, macOS, or Windows with Python 3.12 or newer, pip, and git installed. The pseudonymization key is generated locally and supplied through the environment file, and the dataset is placed in the raw data directory before the first run.

```
git clone https://github.com/soyabeansoupluvr/sales-analytics-dashboard
cd sales-analytics-dashboard
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt
cp .env.example .env # set PSEUDONYM_KEY, add dataset to data/raw/
pytest --cov=src --cov-report=term
streamlit run src/app.py # serves http://localhost:8501
```

The continuous integration pipeline runs the equivalent verification on every push and pull request: flake8 with a 100-character line limit, black in check mode, gitleaks across the full history, and pytest with branch coverage under a 60% failure threshold.